ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Vũ Việt Anh

PHƯƠNG PHÁP MÃ HÓA SAT CHO BÀI TOÁN CHU TRÌNH HAMILTONIAN

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Khoa học máy tính

HÀ NỘI - 2025

VIETNAM NATIONAL UNIVERSITY
UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Vu Viet Anh

SAT ENCODING FOR THE HAMILTONIAN CYCLE PROBLEM

THE BS THESIS

Major: Computer Science

Supervisor: Dr. To Van Khanh

HANOI - 2025

Acknowledgement

First of all, I would like to express my sincere thanks and deep gratitude to Dr. To Van Khanh, who has accompanied and enthusiastically guided and instructed me throughout the time of writing my graduation thesis.

I would also like to thank BS. Kieu Van Tuyen for always supporting me throughout the research process.

I would like to express my sincere thanks to the teachers and lecturers working and teaching at the VNU - University of Engineering and Technology for creating favorable conditions for me to study and research.

Finally, please allow me to thank my family, friends and relatives for always accompanying and encouraging me to complete this thesis.

In the process of researching and completing the thesis, mistakes are inevitable, so I look forward to receiving comments from the reviewers and the council.

Declaration

I am Vu Viet Anh, a student of class K66-CS2, majoring in Computer Science. I hereby declare that the thesis "SAT encoding for the Hamiltonian Cycle Problem" is my own research and development. The research content in the thesis is completely authentic.

The information used in the thesis is well-founded and no content is copied from documents without clearly citing references. I am responsible for this assurance.

Hanoi, May 1, 2025

Student

Vu Viet Anh

Abstract

The Hamiltonian Cycle Problem (HCP) is a fundamental NP-complete problem in graph theory and combinatorial optimization, where the goal is to find a cycle that visits each vertex of a graph exactly once. This problem is crucial for route optimization, network design, and a wide range of practical applications. Despite extensive research, no single method exists that efficiently solves the problem across all graph instances, particularly in dense or large-scale graphs. This variation highlights the importance of using customized methods to handle different types of graph structures, encouraging the use of advanced techniques to make solving the problem more efficient.

This thesis leverages Boolean Satisfiability (SAT) solvers to address the Hamiltonian Cycle Problem, focusing on the Distance encoding framework with a binary adder successor encoding as a foundation. To enhance performance, the research introduces four optimization techniques: an *optimized preprocessing* phase to reduce the search space, *symmetry breaking* to eliminate redundant solutions, a *start vertex* heuristic to guide the solver, and the use of the *PBLib* to efficiently encode exactly-one constraints. The effectiveness of these methods is evaluated through comprehensive experiments on a well-known benchmark dataset.

Keywords: *Constraint programing*, *SAT*, *Hamiltonian cycle*, *Hamiltonian path*

Contents

List of Tables

List of Figures

List of Abbreviations

Abbreviations	Fullform
ALO	At-Least-One
AMO	At-Most-One
BDE	Baseline Distance Encoding
CARD	Cardinality Networks
CNF	Conjunctive Normal Form
CRE	Chinese Remainder Encoding
DE	Distance Encoding
EO	Exactly-One
FHCP	Flinders Hamiltonian Cycle Project
HCP	Hamiltonian Cycle Problem
LFSR	Linear feedback shift register
LSB	Least significant bit
MSB	Most significant bit
ODE	Optimized Distance Encoding
PBLib	Pseudo-Boolean Library
SAT	Boolean Satisfiability Problem
TSP	Travelling Salesman Problem
UNSAT	Unsatisfiable

Preface

Finding efficient solutions to difficult problems, especially in combinatorial optimization and graph theory, continues to be an important area of research in computer science. One well-known example is the Hamiltonian Cycle Problem (HCP), which is NP-complete and difficult to solve because it requires searching through a very large number of possibilities. Traditional algorithms often struggle to handle large instances of this problem. However, recent progress in Boolean Satisfiability (SAT) solving has opened up new possibilities. Modern SAT solvers are very effective at handling complex constraints, making them a useful tool for solving many combinatorial problems when those problems are carefully translated, or encoded, into SAT form.

This thesis focuses on exploring the connection between SAT solving and the Hamiltonian Cycle Problem by improving the way the problem is translated into SAT form. Rather than starting from the beginning, this work builds on previous research and methods that have already shown promising results. In particular, the study centers on enhancing the 'Distance encoding' approach, with a focus on the binary adder successor version introduced by Zhou et al. in 2020 [36]. The main contributions of this thesis include exploring and applying specific optimization techniques. These involve developing a more effective *preprocessing* step to reduce the complexity of the problem in its encoded form; applying additional strategies such as *symmetry breaking* and selecting the minimum-degree vertex as the *starting vertex* to further guide the solver toward faster solutions; and enhancing the encoding of exactly-one (EO) cardinality constraints using the *PBLib*.

The structure of this thesis is organized to guide the reader through the problem, background, proposed methods, and evaluation. Chapter 1 provides an introduction to the Hamiltonian Cycle Problem, discusses its significance, and outlines the context of related work, setting the stage for the investigation. Chapter 2 lays the theoretical groundwork, covering propositional logic, SAT fundamentals, and existing HCP encoding techniques. Chapter 3 details the core contributions – proposed optimization

methods for the Binary adder Distance encoding. Chapter 4 describes the experimental methodology, benchmark datasets, presents the performance results, and provides a comparative analysis. Finally, the last section concludes the thesis, summarizing findings, acknowledging limitations, and suggesting potential avenues for future research. It is hoped that this work provides useful insights and contributes to the broader effort of applying SAT-solving techniques to complex graph problems like the Hamiltonian cycle.

Chapter 1

Introduction

In this chapter, the thesis will introduce Hamiltonian cycle as well as provide typical practical applications of Hamiltonian cycle. Section 1.2 of the thesis will introduce traditional approaches, followed by the SAT Encoding method and its applications so that readers have the most general view of this method.

1.1 Hamiltonian Cycle Problem and applications

1.1.1 Problem definition

The Hamiltonian Cycle Problem [28] is a classic problem in graph theory. Given a graph, the challenge is to determine whether there exists a cycle that visits every vertex exactly once and returns to the starting vertex. Such a cycle is called a Hamiltonian cycle.

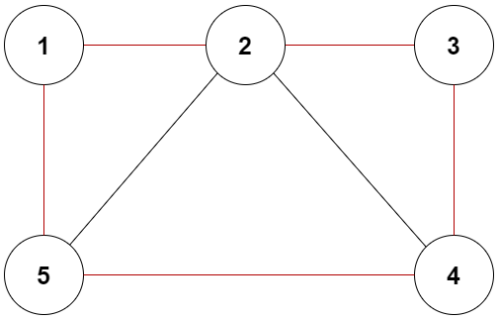


Figure 1.1: The graph has Hamiltonian cycle

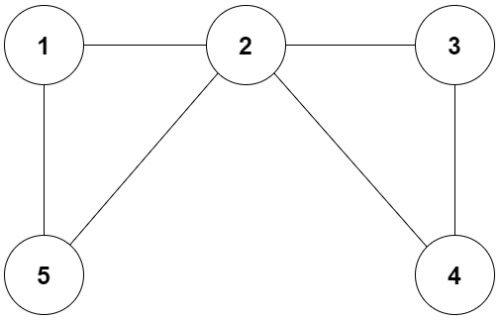


Figure 1.2: The graph has no Hamiltonian cycle

Looking at the two examples above, it is easy to see that the graph in *Figure 1.1* has a Hamiltonian cycle. A visible cycle is in the order $1 \longrightarrow 2 \longrightarrow 3 \longrightarrow 4 \longrightarrow 5$.

The graph in *Figure 1.2* not have a Hamiltonian cycle, because there is no way to go through the vertices exactly once and return to the original vertex.

1.1.2 Applications of Hamiltonian cycle

The Hamiltonian cycle, a fundamental concept in graph theory, holds profound significance in scientific research, spanning mathematics, computer science, biology, chemistry, and computational biology. In mathematics, HCP is a cornerstone of discrete mathematics, serving as a canonical NP-complete problem that challenges researchers to develop efficient algorithms for combinatorial optimization. Its study enhances understanding of graph structures, enabling insights into the connectivity and topological properties of complex systems, such as communication networks or social graphs[25]. In computer science, HCP underpins the development of algorithms for solving optimization problems, with applications in scheduling, network design, and algorithm theory. The problem's NP-completeness [2] drives research into heuristic, approximation, and exact methods, including SAT-based encodings. In computational biology, HCP is instrumental in analyzing molecular structures and gene sequences, where it models sequential constraints in DNA and protein folding [25]. For instance, researchers use HCP to map gene interactions, optimizing the analysis of genetic networks to uncover evolutionary relationships or disease pathways. Similarly, in chemistry, HCP aids in studying molecular configurations, such as the arrangement of atoms in complex compounds [24], facilitating the design of new materials or drugs. These scientific applications often require advanced computational techniques, such as backtracking, dynamic programming, or SAT encodings, to address the exponential complexity of finding Hamiltonian cycles in large graphs. By providing a framework to model and solve sequential and structural problems, HCP remains a critical tool in advancing scientific discovery across these disciplines.

In real-world applications, HCP's versatility is evident in its extensive use across transportation, logistics, electronics, urban planning, and even recreational mathematics, addressing a wide range of optimization and design challenges. A primary application is in solving the Traveling Salesman Problem (TSP) [9], a generalization of HCP, which seeks the shortest route visiting a set of destinations exactly once before returning to the starting point. TSP is critical in transportation for optimizing delivery routes, such as those used by logistics companies like Amazon or postal services, minimizing fuel costs and delivery times. In tourism, HCP informs the design of efficient travel itineraries, enabling tourists to visit multiple cities or landmarks in

a single trip with minimal travel distance [25]. Urban navigation systems [25] also leverage HCP to plan optimal routes through city streets, reducing traffic congestion and improving mobility. In logistics, HCP optimizes the movement of autonomous robots in modern warehouses, where robots must navigate to specific storage locations without redundant paths, enhancing throughput in e-commerce fulfillment centers [21]. Beyond transportation, HCP is pivotal in electronics for designing integrated circuits and printed circuit boards [25], where it optimizes the placement and connection of components to reduce wiring costs and improve signal efficiency. In computer network design [16], HCP ensures robust and efficient routing protocols by modeling network topologies that guarantee connectivity. Additionally, HCP finds applications in scheduling, such as arranging conference seating to satisfy specific constraints (e.g., the round-table problem) or optimizing task sequences in project management [6]. Recreational applications include classic problems like the knight's tour [32] on a chessboard, where a knight visits each square exactly once, and William Rowan Hamilton's "around the world" game [35], which involves finding a Hamiltonian cycle on a dodecahedral graph. These diverse applications highlight HCP's critical role in solving combinatorial optimization problems, driving efficiency, and enabling innovative solutions across industries and everyday life.

1.2 Related works

1.2.1 Traditional approaches

The challenge of finding a Hamiltonian cycle is a well-known NP-complete problem in computer science. Despite significant research, there's no efficient algorithm that can solve this problem for all graph structures. Below are some traditional Exact algorithms and Heuristic/approximation algorithms for HCP.

Brute Force

This problem can be solved using a naive algorithm: This method checks every possible permutation of vertices [22] to see if it forms a Hamiltonian cycle. There will be $n!$ configurations. So the overall time complexity of this approach will be $O(n!)$.

Backtracking

This algorithm explores all possible paths, but it prunes branches [29] that cannot lead to a Hamiltonian cycle. Before adding a vertex, check for whether it is adjacent to the previously added vertex and not already added. If such a vertex is identified,

it is included as part of the solution. The algorithm is more efficient than brute force but still has exponential time complexity.

Dynamic Programming

Another approach is to use dynamic programming to find Hamiltonian paths [12], thereby finding Hamiltonian cycle where the last vertex is a neighbor to the first vertex. The idea is such that for every subset S of vertices, check whether there is a Hamiltonian path in the subset S that ends at vertex v where $v \in S$. If v has a neighbor u, where $u \in S - \{v\}$, therefore, there exists a Hamiltonian path that ends at vertex u. While this approach is more efficient than brute force, it still has exponential time complexity and becomes impractical for large graphs.

Heuristic and approximation algorithms

These algorithms don't guarantee to find an optimal solution but are often used for large-scale problems. The Nearest Neighbor heuristic [17] is a simple approach for finding approximate solutions to problems like the Traveling Salesman Problem. It involves building a solution by sequentially selecting the closest unvisited point. While computationally efficient, it often yields suboptimal results. Its simplicity makes it suitable for quick estimates, but more sophisticated methods are generally preferred for precise solutions.

1.2.2 SAT approaches

Over the years, numerous studies have explored SAT-based approaches to solve HCP. In 2014, Prestwich et al. presented an incremental SAT-based method [30]. This work introduced a naive incremental SAT approach that iteratively constructs partial cycles, using native Boolean cardinality constraints to enforce degree and connectivity requirements. While innovative for its time, the method's incremental nature required multiple solver calls, leading to higher runtime and less efficiency compared to later encodings, particularly for large graphs.

A significant advancement came in 2020 with Zhou et al, who proposed the Distance encoding [36]. This paper proposed the Distance encoding, which uses a compact log-based position encoding to assign vertices to cycle positions, requiring $O(n \log n)$ variables. It also introduced three distinct successor encodings to model the cycle's structure, significantly improving efficiency over prior methods. The Distance encoding's preprocessing and compact clause generation made it a benchmark

for HCP SAT encodings, outperforming earlier approaches like the 2014 incremental method.

More recently, Zhou's 2024 paper [37] introduced the Vertex Elimination Encoding (VEE). VEE simplifies the graph by removing a vertex and mapping a cycle in the reduced graph to one in the original, avoiding explicit no-sub-cycle constraints. However, VEE generates $O(n^4)$ clauses, making it less efficient than the 2020 Distance encoding, though it shows promise for sparse instances when hybridized with Distance encoding.

These works highlight the evolution of SAT-based HCP encodings, from naive incremental methods to Distance encoding and vertex elimination approaches. Ongoing research focuses on the Distance encoding by optimized techniques, integrating advanced libraries for efficient cardinality constraint handling.

Chapter 2

Background

SAT encoding is fundamentally based on Conjunctive Normal Form (CNF) formulas. This chapter presents the mathematical foundations underlying SAT encoding, explores various CNF encoding techniques and their optimizations, and discusses the application of SAT solvers to HCP. In addition, it reviews relevant prior research in this area to provide context for the proposed approaches.

2.1 Propositional logic and CNF formulas

2.1.1 Propositional logic

Propositional logic formulas or propositional expressions are built from variables and the operators AND ($\wedge$), OR ($\vee$), NOT ($\neg$) and parentheses.

Clauses: Each sentence that is stated as True or False is called a clause, denoted by $P, Q, R, ...$

The logical operations employed in this research include the following:

- **Negation:** A proposition that is True when proposition A is False, and False when A is True, is called the negation of A, denoted $\neg A$.
- **Conjunction:** A proposition that is True only when both propositions A and B are True is called the conjunction of A and B, denoted $A \wedge B$.
- **Disjunction:** A proposition that is False only when both propositions A and B are False is called the disjunction of A and B, denoted $A \vee B$.
- **Implication:** A proposition that is False only when proposition A is True and proposition B is False is called the implication from A to B, denoted $A \rightarrow B$.

In other words, an implication is False only when the premise is True and the conclusion is False.

The behavior of these logical operations is summarized in a truth table, which lists all possible combinations of truth values for the input propositions and the corresponding truth values of the resulting proposition for each operation.

Table 2.1: Truth table of logical operations

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \rightarrow B$
0	0	1	0	0	1
0	1	1	0	1	1
1	0	0	0	1	0
1	1	0	1	1	1

2.1.2 CNF formulas

A propositional logic formula is in Conjunctive Normal Form (CNF) if it is a conjunction (AND) of one or more clauses, where each clause is a disjunction (OR) of literals. A literal is a propositional variable or its negation. Every logical proposition can be transformed into a CNF formula based on the transformation principles of propositional logic. The standard form of CNF has the form:

$$(A_1 \vee ... \vee A_n)_1 \wedge ... \wedge (B_1 \vee ... \vee B_n)_p$$

Example: $P = A \wedge (\neg A \vee B)$

$\Rightarrow$ The formula P is SAT because the set of values A = True, B = False result in P = True.

Every propositional formula can be transformed into a CNF formula based on the transformation principles of propositional logic.

2.2 SAT introduction

2.2.1 Definition and application of SAT

The SAT problem is a problem in computer science that tests the satisfiability (SAT) or unsatisfiability (UNSAT) of a propositional logic formula. The SAT problem is a problem that is proven to belong to the class NP - complete, other problems that want to be proven to belong to the class NP - complete can be reduced to the SAT

problem. A propositional logic formula is SAT if there exists a set of True or False values on the propositional logic variables that makes the formula take the value True. Conversely, the formula is UNSAT if and only if every set of True or False values on the propositional logic variables always makes the formula take the value False.

Example 1.2a: *Example of SAT formula:*
Given the Propositional logic formula:

$$F = (x_1 \vee x_2 \vee x_3) \wedge (-x_1 \vee x_2 \vee x_3)$$

where x_1, x_2, x_3 are propositional logic variables.

Formula F is SAT because for the set of values $x_1 =$ True, $x_2 =$ True, $x_3 =$ True , F gives the result True.

Example 1.2b: *Example of UNSAT formula:*
Given the propositional logic formula:

$$F = (-x_1 \vee x_1 \vee -x_2) \wedge (x_1 \vee -x_3) \wedge (x_1 \vee x_2)$$

where x_1, x_2, x_3 are propositional logic variables.

Formula F is UNSAT because for every set of values, F always gives the result False.

Propositional Logic: The input to a SAT problem is a Propositional Logic formula, usually expressed in Conjunctive Normal Form (CNF) or Disjunctive Normal Form (DNF).

Conjunctive Normal Form (CNF): As stated earlier, a CNF formula is a conjunction of clauses, where each clause is a disjunction of literals. The normal form of a CNF union has the following form:

$$TSC_1 \wedge TSC_2 \wedge ... \wedge TSC_n$$

where $TSC_i \equiv (P_1 \vee ... \vee P_m)$ with $m, n > 1$ and P_i is propositional logic variables.
Any propositional logic formula can be converted into a CNF formula by equivalent transformations such as: De Morgan's Law, distributive laws, commutative operations, ...

In addition to SAT Encoding, SAT is used in many fields of information technology. Some typical fields can be mentioned as follows: In the formal method, SAT is used to test hardware models [11], test software models or generate test samples [18]. In the field of artificial intelligence, SAT is used for planning problems, knowledge

introduction problems, and in intellectual games. In the field of automatic design, SAT is used for: equivalence testing, delay calculation, error detection, etc.

2.2.2 SAT solver

A SAT Solver is a program that automatically determines whether a given propositional logic formula is satisfiable (SAT) or unsatisfiable (UNSAT). Nowadays, SAT Solvers are widely interest and developed in the scientific community because of their ability to solve propositional logic formulas with hundreds of thousands of variables and millions of CNF propositions. Every year, the SAT Competition [10] is held in conjunction with prestigious scientific conferences in the world to find the strongest SAT Solvers and announce new algorithms for SAT problems, effective experimental installation techniques in strong SAT solvers. The competition has attracted the attention of the scientific community, attracting SAT Solvers from prestigious universities and research institutes around the world.
Currently, there are many SAT solving tools that can process a large number of clauses and variables to produce results in the most optimal time such as Minisat [31], Lingeling [4], Glucose [3], Rsat [34]...

CaDiCaL[5, 27], developed by Armin Biere, is a high-performance, open-source SAT solver renowned for its efficiency in solving Boolean satisfiability problems. The solver excels in both academic and industrial applications, such as formal verification and combinatorial optimization. Its power was proven by securing first place in the SAT track of the 2019 SAT Race and second place overall.

2.2.3 SAT encoding

SAT Encoding is a method in which some problems can be solved by converting them into SAT problems: Representing problems using Propositional Logic formulas and applying SAT Solver to solve the Propositional Logic formulas.

First, it is necessary to determine the input data or input of the problem to be solved. Then the Encoding encoder receives that input data, determines the rules and encodes those rules into CNF clauses and gives the output as a data file including the number of variables, number of clauses, CNF clauses. SAT Solver will receive the output file of the Encoding encoder as input and process the expressions and generate the output result. The output result can be SAT if SAT Solver finds a data set that satisfies the CNF clauses and UNSAT if it does not find any data set. If the result is SAT, SAT Solver will output another file containing the appropriate data set

that SAT Solver found. From the found data set, it will deduce the solution to the problem and get the result as the answer corresponding to the input data.

Exactly-one constraint

A cardinality constraint [33] is commonly used in optimization problems to limit how many variables in a set of Boolean variables can be assigned the value 1. It is typically written in the form:

$$\sum_{i=1}^{n} x_i \oplus k \quad \text{when } \oplus \in \{\leq, \geq, =\} \quad \text{and } x_i \in \{0, 1\}$$

When $k = 1$ and $\oplus$ is '=', this forms the exactly-one constraint, crucial for problems like HCP to ensure each vertex has exactly one incoming and one outgoing arc. This requires exactly one binary variable in a set $\{x_1, x_2, \ldots, x_n\}$ to be True:

$$\sum_{i=1}^{n} x_i = 1$$

This combines an at-least-one (ALO) condition $\sum_{i=1}^{n} x_i \geq 1$ and an at-most-one (AMO) condition $\sum_{i=1}^{n} x_i \leq 1$. Efficient AMO encodings, such as pairwise, bisect, product, and hybrid, are critical for SAT-based solvers to minimize clause counts.

Pairwise Encoding (PW)

The pairwise encoding for the AMO constraint ($\leq 1(B_1, B_2, \ldots, B_n)$) generates clauses $\neg B_i \vee \neg B_j$ for all $i \in 1..n-1$, $j \in i+1..n$. It ensures at most one variable is True but produces $O(n^2)$ clauses, making it impractical for large n in HCP, where numerous arc variables are involved.

Bisect Encoding (BS)

The bisect encoding, a variant of commander encoding [23], splits variables into two groups: $G_1 = \{B_1, \ldots, B_m\}$, $G_2 = \{B_{m+1}, \ldots, B_n\}$ where $m = \lfloor n/2 \rfloor$. A commander variable T is introduced, with constraints:

- (BS-1) $B_i \Rightarrow T$ for $i \in 1..m$
- (BS-2) $B_i \Rightarrow \neg T$ for $i \in m+1..n$
- (BS-3) $\leq 1(B_1, \ldots, B_m)$

- (BS-4) $\leq 1(B_{m+1}, \ldots, B_n)$

These ensure at most one variable is True across groups, with recursive AMO constraints on each group. BS generates $f(n) = n + 2f(n/2)$ clauses and $g(n) = 1 + 2g(n/2)$ variables, offering better scalability than PW.

Product Encoding (PD)

The product encoding [7] arranges variables in an $m \times m$ matrix, where $m = \lfloor \sqrt{n} \rfloor$, filling extra entries with 0 if n is not a perfect square. It introduces row variables $R_1, \ldots, R_m$ and column variables $C_1, \ldots, C_m$, with constraints:

- (PD-1) $M_{ij} \Rightarrow R_i \wedge C_j$ for $i, j \in 1..m$
- (PD-2) $\leq 1(R_1, \ldots, R_m)$
- (PD-3) $\leq 1(C_1, \ldots, C_m)$

PD generates $f(n) = 2n + 2f(\sqrt{n})$ clauses and $g(n) = 2n + 2g(\sqrt{n})$ variables, making it efficient for large n in HCP by leveraging matrix symmetry.

Hybrid Encoding

The hybrid encoding [36] combines PW, BS, and PD to optimize clause and variable counts. For small $n \leq 4$, PW is used. For larger n, the constraint is divided using BS or PD, selected based on a cost function $f(n) + \alpha g(n)$, where α is a penalty for new variables. For example, with $n = 64$ and $\alpha = 3$, PD first divides into two sub-constraints of 8 variables each, then BS splits these into base constraints encoded with PW.

For a vertex v with outgoing arcs to $u_1, \ldots, u_k$, hybrid encoding ensures that exactly one variable $H_{vu_1}, \ldots, H_{vu_k}$ is True, using PW, BS, or PD depending on the value of k. Similarly, incoming arc variables are encoded to select exactly one incoming arc. This approach efficiently solves the degree constraints in the HCP.

2.3 SAT encoding for solving HCP

To solve the Hamiltonian Cycle Problem using SAT, the problem is transformed into Boolean formulas in CNF. This section presents several encoding methods used in this transformation. The thesis starts with a naive encoding about incremental SAT and then introduces Distance encoding and Vertex Elimination encoding (VEE), each offering different ways to represent the cycle constraints in SAT form.

2.3.1 Incremental SAT-based method

Key idea of the incremental SAT-based method is relaxing connectivity constraints. Instead of encoding all connectivity constraints (which prevent sub-cycles) upfront, the method starts with an initial abstraction that includes only in/out-degree constraints and a constraint to prevent two-node sub-cycles. This significantly reduces the initial number of clauses.

The algorithm 1 presents the *Counterexample-Guided Abstraction Refinement* (CEGAR) loop for HCP [8], which iteratively solves the relaxed SAT problem. If the solution contains sub-cycles, blocking clauses are added to exclude these sub-cycles (both clockwise and counterclockwise). This process repeats until a single Hamiltonian cycle is found or the formula becomes unsatisfiable, indicating no Hamiltonian cycle exists.

Algorithm 1 CEGAR Iteration for solving HCP

```
1: Ψ := initial abstraction of G
2: while Ψ is satisfiable do
3:     if solution contains only one cycle then
4:         return solution                          ▷ Hamiltonian cycle found
5:     end if
6:     Ψ_block := Construct blocking clauses (two for each sub-cycle)
7:     Ψ := Ψ ∧ Ψ_block
8: end while
9: return there is no Hamiltonian cycle
```

Let $G = \langle V, E \rangle$ be a graph with n nodes, and let $A = \{\langle i, j \rangle, \langle j, i \rangle \mid \{i, j\} \in E\}$ be the set of auxiliary arcs. Boolean variables x_{ij} (for $\langle i, j \rangle \in A$) are set to 1 if arc $\langle i, j \rangle$ is included in the cycle, and 0 otherwise. The initial abstraction and iterative refinements use the following constraints:

$$\sum_{\langle i,j\rangle \in A} x_{ij} = 1 \quad \text{for each node } i \in \{1,\ldots,n\} \qquad \text{(out-degree constraint)}$$

$$\sum_{\langle i,j\rangle \in A} x_{ij} = 1 \quad \text{for each node } j \in \{1,\ldots,n\} \qquad \text{(in-degree constraint)}$$

$$x_{ij} + x_{ji} \leq 1 \quad \text{for each edge } \{i,j\} \in E \qquad \text{(two-node sub-cycle prevention)}$$

When a solution contains sub-cycles (e.g., $C_1 : 1 \to 2 \to 3 \to 7 \to 8 \to 1$), blocking clauses are added to prevent them in both directions. For a sub-cycle $(i_1, i_2, \ldots, i_k)$:

$$\neg x_{i_1 i_2} \vee \neg x_{i_2 i_3} \vee \cdots \vee \neg x_{i_k i_1} \qquad \text{(clockwise)}$$

$$\neg x_{i_k i_{k-1}} \vee \neg x_{i_{k-1} i_{k-2}} \vee \cdots \vee \neg x_{i_2 i_1} \qquad \text{(counterclockwise)}$$

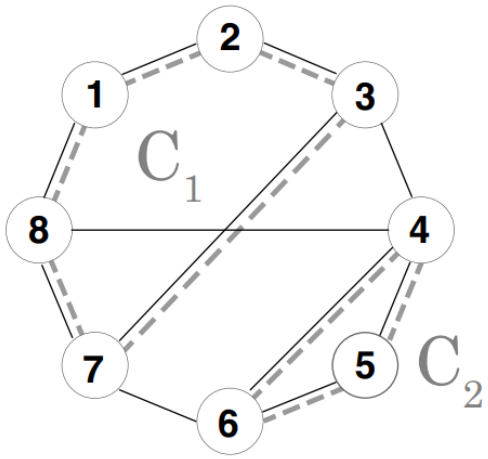


Figure 2.1: Example of subcycles in graph

Example 2.1 for sub-cycle $1 \to 2 \to 3 \to 7 \to 8 \to 1$:

$$\neg x_{12} \vee \neg x_{23} \vee \neg x_{37} \vee \neg x_{78} \vee \neg x_{81} \quad \text{(clockwise)}$$

$$\neg x_{87} \vee \neg x_{73} \vee \neg x_{32} \vee \neg x_{21} \vee \neg x_{18} \quad \text{(counterclockwise)}$$

The incremental SAT-based method reduces the initial clause count, but requires multiple SAT solver calls, potentially increasing runtime for large graphs. In contrast, Distance encoding employs a compact log-based position encoding, resulting in fewer clauses and faster convergence. Zhou's experiments in 2020 research[36] demonstrate that Distance encoding performs better in large instances, making it more effective

for HCP.

2.3.2 Distance encoding

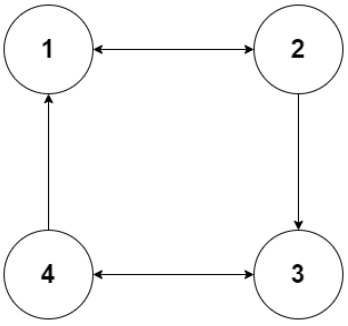
The Distance encoding approach[36] for HCP models the problem as a single-agent path-finding task, where an agent traverses a graph to visit each vertex exactly once, forming a Hamiltonian cycle. This encoding maps each vertex to a distinct position in the cycle, ensuring that the successor relationship between positions prevents sub-cycles. The method has three distinct encodings for the successor function: Unary, Binary adder, and Linear-feedback-shift register (LFSR).

Circuit constraint

In constraint programming (CP), HCP can be represented as global constraint called *circuit* (G) where each node in the graph is assigned a unique position within the cycle.

$$G = [V_1, V_2, ..., V_n]$$

For instance, consider a directed graph in *Figure 2.2* represented by the form of domain variables (or also known as adjacency list) where vertex i is represented by the domain variable V_i (i = 1, 2, 3, 4), and the domain of V_i indicates the outgoing arcs from vertex i. A valuation $V_i = j$ of the domain variables represents a subgraph of G that consists of arcs (i, j) (i ∈ 1 . . . n, j ∈ 1 . . . n).



$V_1 :: [2]$
$V_2 :: [1, 3]$
$V_3 :: [4]$
$V_4 :: [1, 3]$

Figure 2.2: A directed graph and its representation using domain variables

A solution to HCP is an assignment of values to the circuit G such that the resulting sequence of edges forms a complete cycle visiting every vertex exactly once. For example, for the graph in *Figure 2.2*, [2,3,4,1] is a solution because 1 → 2, 2 →

3, 3 → 4, 4 → 1 is a Hamiltonian cycle, but [2,1,4,3] is not because the graph 1 → 2, 2 → 1, 3 → 4, 4 → 3 contains two sub-cycles.

Log encoding

Log encoding [19] is a compact method for representing domain variables in SAT encodings and HCP. Log encoding represents values using a sequence of Boolean variables that encode the magnitude in binary form. If the domain includes both positive and negative values, an additional Boolean variable is used for the sign. For HCP, where positions are positive integers (e.g., 1 to n for a graph with n vertices), only the magnitude is encoded.

In the context of HCP, each vertex i is assigned a position variable P_i, whose domain is the set of possible positions in the cycle (1 to n). The Log encoding uses $\lceil \log_2 n \rceil$ Boolean variables to represent each P_i. For example, for a graph with 5 vertices, 3 Boolean variables ($X_2X_1X_0$) suffice to encode positions 1 to 5. This reduces the number of variables compared to direct encoding, which would require n variables per vertex.

Distance encoding

The Distance encoding employs an $n \times n$ matrix of Boolean variables H, where n is the number of vertices. Each H_{ij} (for $i, j \in \{1, \ldots, n\}$) is 1 if the arc (i, j) is in the Hamiltonian cycle, and 0 otherwise. For arcs (i, j) not present in the input graph G, H_{ij} is set to 0, so the number of variables in H equals the number of arcs in G.

The following channeling constraints connects H to domain variables $[V_1, \ldots, V_n]$, where V_i is the successor of vertex i.

$$H_{ij} \Leftrightarrow V_i = j \quad \text{for } i, j \in \{1, \ldots, n\},\ i \neq j$$

Since each V_i has one value, this ensures each vertex has one outgoing arc. When interpreted in this manner and translated into propositional logic for the SAT solver, the degree constraint can be formally expressed as follows:

$$\sum_{j=1}^{n} H_{ij} = 1 \quad \text{for } i \in \{1, \ldots, n\} \tag{1}$$

Completely similar, the following degree constraints ensure that each vertex has exactly one incoming arc:

$$\sum_{i=1}^{n} H_{ij} = 1 \quad \text{for } j \in \{1, \ldots, n\} \tag{2}$$

To avoid sub-cycles, each vertex i is assigned a unique position $p(i)$, with $s(p)$ as the successor function and $s^k(p)$ as the k-th successor. Vertex 1 is fixed at position 1, and constraints ensure the cycle starts and ends at 1. If arc $(1, i)$ is in the cycle, i is at the successor of position 1, and if arc $(i, 1)$ is in the cycle, i is at the $(n-1)$-th successor.

$$H_{1i} \Rightarrow p(i) = s(1) \quad \text{for } i \in \{2, \ldots, n\} \tag{3}$$

$$H_{i1} \Rightarrow p(i) = s^{n-1}(1) \quad \text{for } i \in \{2, \ldots, n\} \tag{4}$$

Finally, the formula ensures vertices are positioned sequentially in the cycle. This guarantees that if arc (i, j) is in the cycle, j's position follows i's.

$$H_{ij} \Rightarrow p(j) = s(p(i)) \quad \text{for } i, j \in \{2, \ldots, n\},\ i \neq j \tag{5}$$

Three encodings for the successor function

Within the constraint above, the constraint (1) and (2) are easily implemented by a boolean matrix $H[i][j]$, where $H[i][j] = 1$ if the arc $(i,\ j)$ in the Hamiltonian solution. Besides, there are several different ways to encode the successor function $p(j) = s(p(i))$. The 2020 research[36] has proposed three such encodings, namely, the Unary encoding, the Binary adder encoding, and the Linear-feedback-shift-register (LFSR) encoding.

i. Unary encoding

The Unary encoding [36] of the successor function employs a matrix U of Boolean variables of size $n \times n$, where $U_{ip} = 1$ iff vertex i's position is p for $i \in$ 1..n and $p \in$ 1..n. Since vertex 1 is visited first, U_{11} is initialized to 1.

The formulas (1) and (2) are reused at Distance encoding. The other formulas above are converted to the following:

$$H_{1i} \Rightarrow U_{i2} \quad \text{for } i \in \{2, \ldots, n\} \tag{3'}$$

$$H_{i1} \Rightarrow U_{in} \quad \text{for } i \in \{2, \ldots, n\} \tag{4'}$$

$$H_{ij} \wedge U_{ip} \Rightarrow U_{j(p+1)} \quad \text{for } i,j \in \{2,\ldots,n\},\ i \neq j, p \in 2,\ldots,(n-1) \tag{5'}$$

$$\sum_{p=1}^{n} U_{ip} = 1 \quad \text{for } i \in \{1,\ldots,n\} \tag{6}$$

ii. Binary adder encoding

The Binary adder encoding [36] of the successor function employs a log-encoded domain variable P_i for each vertex i, whose domain is the set of possible positions for the vertex. Log encoding requires $\lceil \log_2 n \rceil$ Boolean variables per vertex.

Since vertex 1 is visited first, $P_1 = 1$. Suppose P_1 is in log encoded form $X_{m-1}...X_1X_0$, then $X_0 = 1, X_1 = 0, X_2 = 0, ..., X_{m-1} = 0$. Constraints (3)-(5) are rewritten as:

$$H_{1i} \Rightarrow P_i = 2 \quad \text{for } i \in \{2,\ldots,n\} \tag{3"}$$

$$H_{i1} \Rightarrow P_i = n \quad \text{for } i \in \{2,\ldots,n\} \tag{4"}$$

$$H_{ij} \Rightarrow P_j = P_i + 1 \quad \text{for } i \neq j \in \{2,\ldots,n\} \tag{5"}$$

The successor function $P_j = P_i + 1$ is encoded using an optimized incrementor that eliminates carry variables, reducing the clause count. The encoding processes two bits at a time for higher bits, reducing the clause count by one per two bits, and treats the top four bits as a single unit, further reducing clauses to 21 for those bits.

Let $\langle X_{m-1}X_{m-2}\ldots X_0\rangle$ denote the binary encoding of vertex X, and $\langle Y_{m-1}Y_{m-2}\ldots Y_0\rangle$ denote the encoding of vertex Y, such that $Y = X + 1$ (modulo the cycle length). The encoding incrementally constructs the binary representation of Y by simulating an unsigned addition operation on the bits of X. For the least significant bit (bit position 0), the operation is straightforward: $Y_0 = \neg X_0$, which translates into two clauses in CNF. At bit position 1, the encoding must account for a potential carry from the previous bit. If X_0 is True (i.e., a carry is generated), then $Y_1 = \neg X_1$; otherwise, $Y_1 = X_1$. This logic is captured using four clauses.

For higher bit positions $i > 1$, the encoding continues this pattern recursively. When a carry occurs from the previous bit — specifically, when $\neg Y_{i-1} \wedge X_{i-1}$ is True — the result bit is the negation of the current input bit: $Y_i = \neg X_i$. If the carry persists and both X_i and X_{i-1} are True, then the carry continues to the next position, leading to $Y_{i+1} = \neg X_{i+1}$. In all other cases where no carry is propagated, each output bit Y_i remains equal to the corresponding input bit X_i, maintaining consistency with the logic of binary addition.

iii. LFSR encoding

The LFSR encoding [14, 20, 36] also uses Log encoding for position variables but employs a Linear-feedback-shift register to define the successor function. An LFSR generates a sequence of positions using a feedback mechanism, which can differ from the natural sequence $(1, 2, ..., n)$. The successor function is encoded such that $p(j) = s(p(i))$, where s is determined by the LFSR configuration.

While LFSR encoding is compact, generating fewer clauses than the Unary encoding, it is less effective than the Binary adder encoding when preprocessing is applied. The LFSR encoding struggles with domains that have scattered holes after preprocessing, requiring more prime implicants to cover the domain, whereas the Binary adder encoding benefits from concentrated holes or narrowed domain bounds [36].

Preprocessing

The preprocessing technique [36] enhances the efficiency of the Distance encoding by ruling out infeasible vertex-position assignments before SAT solving. The technique leverages the path-finding model of the HCP, where the agent starts at vertex 1 at time 1, visits each vertex exactly once, and returns to vertex 1 at time $n + 1$. The key idea is to exclude positions for vertices that cannot be reached at specific times based on the graph's structure.

If there are no paths of length $t-1$ from vertex 1 to vertex i, then vertex i cannot be at position t. In Unary encoding, U_{it} is set to 0; in binary encoding, the value $s^{t-1}(1)$ is excluded from P_i's domain. If there are no paths of length $n-t+1$ from vertex i to vertex 1, then vertex i cannot be at position t.

The *shortest-distance heuristic* is used. If the shortest distance from vertex 1 to vertex i is t, then vertex i cannot be at positions 1 to t. If the shortest distance from vertex i to vertex 1 is t, then vertex i cannot be at positions $n-t+2$ to n. Additionally, for undirected graphs where vertex 1 has exactly two neighbors, the agent must visit one neighbor at time 2 and the other at time n, ruling out positions 3 to $n-1$ for both neighbors.

For example, consider a graph where the shortest distance from vertex 1 to vertex X is $t = 3$. This implies that vertex X cannot be placed in positions 1, 2, or 3. Let $P_x = \langle X_{m-1}X_{m-2}\dots X_0\rangle$ denote the position variable for vertex X in the Binary adder encoding, where P_x is a binary representation of its position (from 1 to n). To enforce this constraint, clauses are generated to ensure $P_x \neq 1$, $P_x \neq 2$, and $P_x \neq 3$. The encoding can be expressed as following:

$$X_{m-1} \vee X_{m-2} \vee \cdots \vee X_2 \vee X_1 \vee -X_0 \quad \text{for } P_x \neq 1$$

$$X_{m-1} \vee X_{m-2} \vee \cdots \vee X_2 \vee -X_1 \vee X_0 \quad \text{for } P_x \neq 2$$

$$X_{m-1} \vee X_{m-2} \vee \cdots \vee X_2 \vee -X_1 \vee -X_0 \quad \text{for } P_x \neq 3$$

This results in $\mathcal{O}(n)$ clauses, as there are $t \leq n$ positions to exclude, and each clause has a length of $\log(n)$ due to the binary encoding of positions. The preprocessing technique significantly reduces the search space by fixing variables or narrowing domains, particularly benefiting the Binary adder encoding. This study proposes a method to optimize preprocessing for Binary adder encoding presented in *section 3*, which helps reduce the number of clauses and increase the efficiency of the problem.

2.3.3 Vertex elimination encoding

The *Vertex Elimination Encoding* (VEE) [37] is a SAT encoding for the HCP that leverages the vertex elimination technique to simplify graphs while preserving Hamiltonian cycle properties. VEE maps a Hamiltonian cycle in a reduced graph G', obtained by eliminating a vertex v from the original graph G, to a Hamiltonian cycle in G. It is particularly effective for sparse graphs but generates large encodings for dense graphs, requiring $\mathcal{O}(n^3)$ variables and $\mathcal{O}(n^4)$ clauses in the worst case.

The vertex elimination synthesize a subgraph $H_G = (H_V, H_E)$ such that:

$$H_V = \{v \mid v \in V,\ b_v = 1\}$$

$$H_E = \{(u, v) \mid (u, v) \in E,\ b_{uv} = 1\}$$

Let k be the cardinality of H_V: $k = \sum_{v \in V} b_v$
VEE operates on a directed graph $G = (V, E)$, where a vertex v is eliminated to produce a reduced graph$G = (V, E) \rightarrow G' = (V', E')$

$$\begin{aligned} V' &= V \setminus \{v\} \\ E' &= E \setminus \{(u, v) \mid (u, v) \in E\} \\ &\quad \setminus \{(v, w) \mid (v, w) \in E\} \\ &\quad \cup \{(u, w) \mid u \in \text{nbs}^-(v),\ w \in \text{nbs}^+(v),\ u \neq w\} \end{aligned}$$

where

$$\begin{aligned} \text{nbs}^-(v) &= \{u \mid (u, v) \in E\}, \\ \text{nbs}^+(v) &= \{w \mid (v, w) \in E\} \end{aligned}$$

The following constraints ensure the correct mapping of a Hamiltonian cycle in G to G':

$$k > 1 \wedge b_v \rightarrow \sum_{(u,v)\in E} b_{uv} = 1 \qquad \text{for each } v \in V \qquad \text{(VE-1)}$$

$$k > 1 \wedge b_v \rightarrow \sum_{(v,w)\in E} b_{vw} = 1 \qquad \text{for each } v \in V \qquad \text{(VE-2)}$$

$$k' > 1 \rightarrow \neg b_{uv} \vee \neg b_{vu} \qquad \text{for each } (u,v) \in E \text{ if } (v,u) \in E \qquad \text{(VE-3)}$$

$$b'_{uw} \rightarrow b_{uv} \wedge b_{vw} \qquad \text{for each } (u,w) \in E' \setminus E \qquad \text{(VE-4)}$$

$$\sum_{(u,w)\in E'\setminus E} b'_{uw} \leq 1 \qquad \text{(VE-5)}$$

$$b_{uv} \wedge b_{vw} \rightarrow b'_{uw} \qquad \text{for each } (u,v) \in E, (v,w) \in E, u \neq w \qquad \text{(VE-6)}$$

$$b_{uv} \wedge b_{vw} \rightarrow \neg b_{uw} \qquad \text{for each } (u,v) \in E, (v,w) \in E, u \neq w \qquad \text{(VE-7)}$$

$$\neg b_{uv} \vee \neg b_{vw} \rightarrow b_{uw} = b'_{uw} \qquad \text{for each } (u,v) \in E, (v,w) \in E, u \neq w \qquad \text{(VE-8)}$$

The VEE reduces sparse graphs effectively but generates large encodings $\big(\mathcal{O}(n^4)$ clauses$\big)$, causing timeouts on dense instances. In contrast, Distance encoding uses compact log-based position encoding $\big(\mathcal{O}(n \log n)$ variables$\big)$ and preprocessing, achieving fewer clauses and faster convergence. Zhou's experiments in 2024 research [37] show that Distance encoding solves most benchmarks faster, especially in the FHCP benchmark. That is why this research focuses on improving the Distance encoding.

Chapter 3

Optimized SAT encoding for HCP

This chapter presents the proposed optimization methods designed to enhance the efficiency of SAT encodings for HCP, building upon the Distance encoding. It begins with an overall description of the four key techniques, followed by a detailed exploration of each method to clarify their implementation and impact.

3.1 Overview of optimized method

As discussed in the previous section, Binary adder encoding outperforms Unary and LFSR-based encodings in representing successor functions for the HCP. Therefore, this thesis focuses on optimizing the Binary adder encoding to improve the efficiency and scalability of SAT-based solutions for HCP.

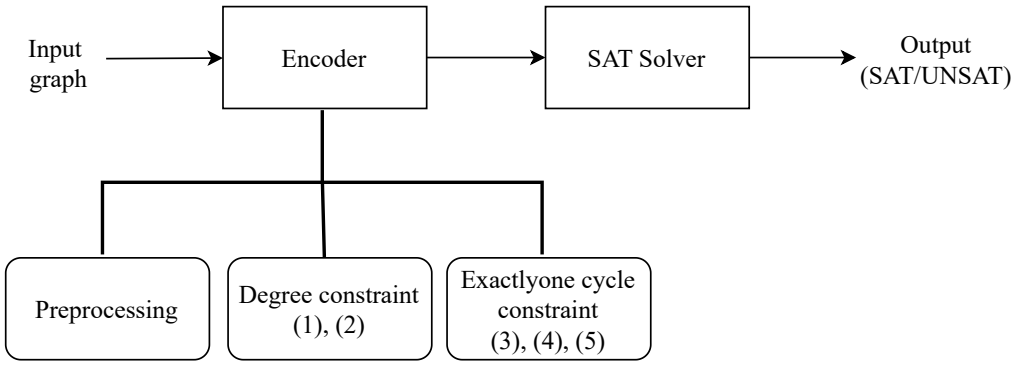


Figure 3.1: The flow of encoding and solving HCP by SAT solver

The Figure 3.1 present the general flow of the SAT-based approach for solving the

HCP. The encoder is responsible for transforming the input graph into a Conjunctive Normal Form (CNF) formula that a SAT solver can process. This encoding process consists of key components, including preprocessing, degree constraints (formular 1, 2), exactly-one-cycle constraint(formular 3, 4, 5)

After encoding, the CNF formula is passed to a SAT solver. The solver searches for a satisfying assignment (model). If such an assignment exists, the solver outputs SAT, indicating that a Hamiltonian cycle exists in the input graph. Otherwise, it outputs UNSAT, confirming that no such cycle exists.

In addition to optimizing the encoding structure, this thesis proposes a method for encoding the max-min constraints used in Log encoding. These constraints are applied during the preprocessing step to further narrow the search space.

For undirected graphs, symmetry-breaking clauses are introduced to reduce redundant solutions. Specifically, the clause $-e_{s,u} \vee -e_{v,s}$ (where u > v) is used to prevent the solver from exploring both directions of the same cycle path, which would otherwise be equivalent due to the undirected nature of the graph.

Furthermore, the encoding selects the starting vertex based on the one with the smallest degree. This heuristic is based on the idea that beginning the reasoning process from a vertex with fewer connections often leads to more effective pruning of the search space.

Finally, the encoding employs the *PBLib* for implementing the "at-most-one" constraint. Compared to traditional approaches such as the Bisect or Product encoding, even the Hybrid encoding, the use of *PBLib* provides higher efficiency and better performance in practice.

3.2 Optimized preprocessing

To improve the performance of the Binary adder encoding, the research propose several optimizations in the preprocessing stage. These optimizations aim to reduce the number of clauses generated and help the SAT solver eliminate infeasible paths early in the search process. This section introduces the concept of max-min constraints for Log encoding and presents a novel preprocessing technique for the Binary adder method.

3.2.1 Max-min constraint for Log encoding

This thesis proposed the max-min constraint for Log encoding that enables comparisons between a binary-encoded integer variable and a constant. It allows to express that a variable (encoded in binary) must be greater than or equal to ($\geq$) or less than or equal to ($\leq$) a constant value.

Let x be an integer variable encoded in binary as $(x_{m-1}, x_{m-2}, \ldots, x_1, x_0)$, where $m = \lceil \log_2 n \rceil$, x_0 is the least significant bit (LSB), and x_{m-1} is the most significant bit (MSB). A constant c is represented in binary as $c_{\text{bin}} = (b_{m-1}, b_{m-2}, \ldots, b_0)$, where $b_i \in \{0, 1\}$, and m bits are used to match the encoding of x.

Encoding $x \leq c$: To ensure $x \leq c$, clauses are constructed to exclude values of x greater than c. For each bit position $i \in [0, m-1]$ from MSB to LSB, if $b_i = 0$, add the clause:

$$\neg x_i \vee \bigvee_{j>i} (X_j \neq b_j) \quad \text{where} \quad X_j = \begin{cases} x_j & \text{if } b_j = 1 \\ \neg x_j & \text{if } b_j = 0 \end{cases}$$

In CNF form, this becomes:

$$\neg x_i \vee \bigvee_{j>i} (\neg L_j) \quad \text{where} \quad L_j = \begin{cases} x_j & \text{if } b_j = 1 \\ \neg x_j & \text{if } b_j = 0 \end{cases}$$

No constraint is added if $b_i = 1$.

Encoding $x \geq c$: To ensure $x \geq c$, clauses are constructed to exclude values of x less than c. For each bit position $i \in [0, m-1]$, if $b_i = 1$, add the clause:

$$x_i \vee \bigvee_{j>i} (X_j \neq b_j) \quad \text{where} \quad X_j = \begin{cases} x_j & \text{if } b_j = 1 \\ \neg x_j & \text{if } b_j = 0 \end{cases}$$

In CNF form, this becomes:

$$x_i \vee \bigvee_{j>i} (\neg L_j) \quad \text{where} \quad L_j = \begin{cases} x_j & \text{if } b_j = 1 \\ \neg x_j & \text{if } b_j = 0 \end{cases}$$

No constraint is added if $b_i = 0$.

For instance, consider encoding constraints $x \leq 5$ and $x \geq 5$ for a variable x represented by 3 bits (x_2, x_1, x_0). The constant $c = 5$ in binary is $c_{\text{bin}} = (b_2, b_1, b_0) = (1, 0, 1)$.

For $x \leq 5$, examine each bit position. At bit 2 (MSB), $b_2 = 1$, so no constraint is needed. At bit 1, $b_1 = 0$, yielding the clause:$\neg x_2 \vee \neg x_1$. At bit 0, $b_0 = 1$, so no constraint is added. The resulting clause $\neg x_2 \vee \neg x_1$ excludes values such as 6 (110) and 7 (111).

For $x \geq 5$, examine each bit position. At bit 2, $b_2 = 1$, yielding the clause x_2. At bit 1, $b_1 = 0$, so no constraint is needed. At bit 0, $b_0 = 1$, yielding the clause $\neg x_2 \vee x_1 \vee x_0$. The resulting clauses, x_2, which excludes values 0 (000), 1 (001), 2 (010), 3 (011), and clause $\neg x_2 \vee x_1 \vee x_0$, which excludes value 4 (100).

The max-min constraint for Log encoding is highly efficient, requiring $O(m) = O(\log_2 n)$ clauses to encode comparisons like $x \leq c$ or $x \geq c$, where m is the number of bits. In contrast, traditional exclusion methods present in section 2.3.2, which enumerate and exclude each invalid value, require $O(n)$ clauses to cover the same range.

3.2.2 Proposed preprocessing

In the original preprocessing method for the HCP present in section 2, vertices are excluded from certain positions in the cycle based on path-length feasibility. While this strategy works effectively with Unary encoding (e.g., setting $U_{i,t} = 0$ for invalid positions), applying it directly in binary-based encodings such as the **Binary adder encoding** is less straightforward. Traditionally, binary encoding applies a "not-equal" ($\neq$) constraint to eliminate invalid positions, which involves a disjunction of all possible encodings except the one to be excluded. This becomes expensive in both clause count and propagation efficiency, especially when many invalid positions must be removed. For example, when the shortest distance from the vertex i to vertex 1 is t, the preprocessing encodes $P_i \neq 2, P_i \neq 3, ..., P_i \neq t$.

To improve performance and reduce the number of clauses, the thesis propose replacing the multiple "not-equal" constraints with two general inequalities: a minimum constraint and a maximum constraint, encoded using the **Max-Min constraint for Log encoding** method.

Let P_i be the binary variable vector representing the position of vertex i in the Hamiltonian cycle. The following two constraints are derived from the shortest path distances in the graph:

- Let $d_1(i)$ be the shortest distance from vertex 1 to vertex i. Then:

$$P_i \geq d_1(i) + 1$$

This ensures that vertex i cannot appear earlier than $d_1(i) + 1$ in the cycle.

- Let $d_i(1)$ be the shortest distance from vertex i to vertex 1. Then:

$$P_i \leq n - d_i(1) + 1$$

This prevents vertex i from being placed too late in the cycle, since the agent must return to vertex 1 in $d_i(1)$ steps.

Both constraints are encoded using the Log encoding max/min formula, which leverages binary representations of the bounds and recursively applies constraints on the bits of P_i. This encoding is significantly more compact than listing all forbidden positions using inequality chains.

For example, a graph with 8 vertices, and vertex 5 has the following distances. Shortest distance from vertex 1 to vertex 5: $d_1(5) = 3$ and shortest distance from vertex 5 to vertex 1: $d_5(1) = 4$. Then, in preprocessing, enforce:

$$P_5 \geq 4 \quad \text{(i.e., vertex 5 cannot appear in positions 1–3)}$$

$$P_5 \leq 5 \quad \text{(i.e., vertex 5 cannot appear in positions 6–8)}$$

This reduces the possible positions of vertex 5 to just positions 4 and 5. Instead of encoding the exclusions $P_5 \neq 1, P_5 \neq 2, P_5 \neq 3, P_5 \neq 6, \ldots$ so encode the two bounds compactly using binary logic constraints. This saves both variables and clauses in the final CNF formula.

This preprocessing technique is tailored specifically for the Binary adder encoding, leveraging its ability to handle binary representations naturally and compactly. When integrated, this optimization improves the efficiency of solving HCP instances, particularly in large or sparse graphs.

3.3 Symmetry breaking

In SAT encodings for HCP, with the undirected graph, symmetry breaking [15] plays an important role in reducing the search space of the SAT solver. The main goal is to eliminate redundant solutions that are symmetric (i.e., equivalent under some transformation) so that the solver does not waste time exploring them. A Hamiltonian cycle is cyclic and undirected, meaning that the reverse direction of a cycle is also considered the same.

For example, in a 4-node graph, the cycle $1 \to 2 \to 3 \to 4 \to 1$ and the cycle $1 \to 4 \to 3 \to 2 \to 1$ are equivalent. Without symmetry breaking, both would be accepted as different solutions by the SAT solver, even though they represent the same Hamiltonian cycle. This leads to inefficiency because the solver spends time exploring redundant possibilities.

To remove symmetry in undirected graphs, the research can use the following simple symmetry-breaking clause:

$$\neg H_{s,u} \vee \neg H_{v,s} \quad \text{for } u > v$$

Where:

- $H_{i,j}$ is a Boolean variable indicating whether there is a directed edge from vertex i to vertex j in the Hamiltonian cycle.
- s is the starting vertex of the cycle
- u, v are two vertices connected to s, and suppose $u > v$.

This clause says: if both $H_{s,u}$ and $H_{v,s}$ are True (i.e., the cycle goes from s to u, and then comes back from v to s), then u must not be greater than v. This eliminates one of the two symmetric paths that start from s in opposite directions.

3.4 Encoding for start vertex of cycle

In the HCP, a cycle visits each vertex exactly once, forming a closed path. The rotational symmetry of the cycle, which allows any vertex to be the starting point, generates redundant solutions, expanding the search space of the SAT solver. To eliminate these symmetries and improve the efficiency of solving, the start vertex is fixed to a specific vertex $s \in V$, chosen as the vertex of the minimum degree. This heuristic leverages the fact that low-degree vertices are more constrained, reducing the search space by prioritizing their placement early in the cycle.

In Binary adder encoding, each vertex $i \in V$ is assigned a unique position in the cycle using a binary variable vector $P_i = (x_{i,m-1}, x_{i,m-2}, \ldots, x_{i,0})$, where $m = \lceil \log_2 n \rceil$ bits represent positions from 1 to n. To designate the minimum-degree vertex s as the start (position 1), the binary representation of P_s is set to 1. This is enforced with unit clauses, ensure $P_s = 1$:

$$\{\neg x_{s,j} \text{ for } j \in [1, m-1], \quad x_{s,0}\}$$

The cycle's structure is maintained with the following successor constraints for the start vertex s, the constraints 3", 4", 5" are transformed as follows:

$$H_{si} \Rightarrow P_i = 2 \qquad \text{for } i \in V, i \neq s$$
$$H_{is} \Rightarrow P_i = n \qquad \text{for } i \in V, i \neq s$$
$$H_{ij} \Rightarrow P_j = P_i + 1 \qquad \text{for } i, j \in V, \{i, j\} \neq s, i \neq j$$

For example, a graph with $n = 7$ vertices ($m = 3$), let vertex s, the minimum-degree vertex, be fixed at position 1 ($P_s = 001$). The 3 clauses are $\neg x_{s,2} \wedge \neg x_{s,1} \wedge x_{s,0}$. If $H_{s,4} = 1$, then $P_4 = 2 = 010$, encoded as: $\neg x_{4,2} \wedge x_{4,1} \wedge \neg x_{4,0}$. This ensures vertex 4 follows s, with similar constraints applied to other vertices to maintain the cycle's sequence.

3.5 Using PBLib Library

In HCP, the exactly-one constraint is essential for enforcing degree constraints. Optimizing this constraint is critical for improving solver efficiency. The *PBLib Library*[26] is a robust tool for encoding pseudo-boolean (PB) constraints into SAT that provides methods like `encode_at_least_k` and `encode_at_most_k` to efficiently implement the exactly-one constraint.

The exactly-one constraint ($\sum_{i=1}^{n} x_i = 1$) combines an at-least-one (ALO) condition ($\sum_{i=1}^{n} x_i \geq 1$) and an at-most-one (AMO) condition ($\sum_{i=1}^{n} x_i \leq 1$). The degree constraints, as defined in Formulas 1 and 2, enforce this for each vertex. These constraints ensure that each vertex participates in exactly one incoming and one outgoing arc, forming a valid Hamiltonian cycle.

$$\sum_{j=1}^{n} H_{ij} = 1 \quad \text{for } i \in \{1, \ldots, n\}$$

$$\sum_{i=1}^{n} H_{ij} = 1 \quad \text{for } j \in \{1, \ldots, n\}$$

The *PBLib* facilitates encoding the exactly-one constraint by combining `encode_at_least_k` and `encode_at_most_k` with $k = 1$. *PBLib* employs techniques like sequential counters or cardinality networks to achieve more efficient encodings with fewer auxiliary variables and clauses. In this research, the `Cardinality network` is used for the at-most-one (AMO) encoding.

While traditional pairwise encodings generate $\mathcal{O}(k^2)$ clauses for a vertex with k neighbors, *PBLib*'s encoding leverage advanced techniques to achieve $\mathcal{O}(k \log k)$ clauses [26] in some configurations as *CARD*, significantly enhancing scalability. This optimization reduces the encoding size and improves solver performance, enabling faster convergence for HCP instances.

Chapter 4

Experiment and Result

This chapter evaluates the performance of the proposed optimization methods for solving HCP using SAT encodings, compares the optimized Distance encoding against its non-optimized counterpart and the Chinese Remainder Encoding, and provides a comparative analysis to highlight the impact of the proposed optimizations.

4.1 Experimental environment

In this study, the SAT solver *CaDiCaL*[5] is used for evaluating the CNF encodings of the HCP. *CaDiCaL* is a modern conflict-driven clause learning (CDCL) SAT solver known for its clean architecture, rich feature set, and strong performance in both standalone and incremental solving tasks. The recent tool paper by Biere et al. provides a detailed overview of the solver's design and capabilities. The authors highlight *CaDiCaL*'s effectiveness as a general-purpose SAT solver while maintaining a focus on clarity, modularity, and extensibility. Its architecture has made it a popular choice not only for end users but also for researchers developing new SAT techniques or integrating SAT solving into larger systems such as SMT solvers.

The environmental configuration details of the experimental computer are described in the Table 4.1.

Table 4.1: Configuration of the experimental environment

Processor	Debian GNU/Linux
RAM	64 GB
CPU	8 vCPU, 4 cores
Language	Python version 3.9
SAT Solver	Cadical version 1.95
Timeout	600s

4.2 Benchmark dataset

To evaluate the effectiveness and scalability of SAT-based encoding methods for the HCP, the thesis use the FHCP Challenge Set [13] — a widely recognized and challenging benchmark suite in the HCP research community, which is the HCP benchmark used in the 2019 XCSP solver competition[1].

The FHCP Challenge Set, introduced by Haythorpe in 2018, is a collection of 1001 Hamiltonian graphs specifically designed to be difficult for standard HCP heuristics. The set was initially created to test the limits of exact HCP solvers and remains a standard benchmark for evaluating new algorithms and encodings.

Recent research papers, including 2020's research[36] and 2021's research [15], also use the FHCP benchmark for experimental validation. This thesis select the same 10 instances from the FHCP Challenge Set that were used in two above paper, bonus 8 large graphs used in Chinese Remainder Encoding research, with vertex counts ranging from 338 to 3132. By using the same dataset, this thesis enables a fair and direct comparison with prior work, allowing to assess the competitiveness of proposed SAT encodings under similar conditions.

Table 4.2: The selected instances from benchmark dataset FHCP

graph#	**Number of vertices |V|**	**Number of edges |E|**
48	338	776
162	909	206571
171	996	1495
197	1188	1783
223	1386	2268
237	1476	2215
249	1558	2338
252	1572	2359
254	1582	2374
255	1584	2799
424	2466	4240
446	2557	4368
470	2740	4509
491	2844	4267
506	2964	4447
522	3060	4591
526	3108	4663
529	3132	4699

4.3 Experimental results

To evaluate the effectiveness of the proposed encoding strategies for the HCP, experiments were carried out using the selected FHCP benchmark set. A total of 16 configurations were created by combining four different optimization techniques: preprocessing optimization, symmetry breaking, start vertex selection and usage of *PBLib*. Each configuration is tested on 18 graphs from the FHCP Challenge Set. The results in the Appendix section fully represent the runtime (in seconds) for each configuration across these instances. The best-performing configuration is the one where all four optimization methods are applied.

The 2021's paper [15] is a research that proposed a compact and efficient SAT encoding method for the HCP, combining ideas from incremental SAT solving, Binary adder encoding, and LFSR encoding. By representing vertex positions in the cycle using residues modulo small prime numbers, *CRE* enables the formulation of smaller and more efficient SAT instances. In this experiment, three configurations were compared: the optimized Distance encoding (*ODE*), the baseline Distance encoding without optimization (*BDE*), and the *CRE*. While *CRE* often requires summing runtimes over multiple cycle lengths due to its need to iteratively try different cycle lengths before finding a valid Hamiltonian cycle, both *ODE* and *BDE* use a fixed Binary adder structure.

Table 4.3: Overall time comparison (in seconds)

graph#	BDE	ODE	CRE
48	57.693	**46.573**	87.976
162	283.664	**21.672**	46.89
171	14.646	6.182	**2.234**
197	22.269	10.213	**4.373**
223	47.119	**26.523**	81.985
237	15.171	*11.862*	**10.694**
249	39.848	33.517	**0.999**
252	29.047	*19.596*	**17.244**
254	31.552	13.11	**0.911**
255	**16.494**	70.305	34.535
424	*timeout*	*timeout*	*timeout*
446	*timeout*	*timeout*	*timeout*
470	*timeout*	*timeout*	*timeout*
491	130.744	111.16	**23.811**
506	124.69	111.77	**24.35**
522	142.307	88.375	**33.942**
526	129.293	111.288	**39.417**
529	133.838	126.74	**99.902**

The results in Table 4.3 demonstrate that the optimized Distance encoding (*ODE*) consistently outperforms the non-optimized version (*BDE*) across most benchmark graphs. For instance, in graph 162, the optimized encoding achieves a runtime of 21.672 seconds, significantly faster than the non-optimized version's 283.664 seconds. Similarly, in graphs 48, 223, 491, 506, 522, 526, and 529, the optimized encoding reduces runtimes, with improvements ranging from modest (e.g., 126.74 vs. 133.838 seconds in graph 529) to substantial (e.g., 26.523 vs. 47.119 seconds in graph 223). The only exception is graph 255, where the non-optimized version is faster (16.494 vs. 70.305 seconds), possibly due to specific graph properties affecting the solver's branching heuristics. For graphs 424, 446, and 470, all encodings timed out, indicating these instances remain challenging regardless of optimization.

When compared to the *CRE*, the optimized Distance encoding is highly competitive. It outperforms *CRE* in several graphs, such as 48 (46.573 vs. 87.976 seconds), 162 (21.672 vs. 46.89 seconds), and 223 (26.523 vs. 81.985 seconds), demonstrating superior efficiency in these cases. In graphs like 237 (11.862 vs. 10.694 seconds) and 252 (19.596 vs. 17.244 seconds), *CRE* is slightly faster, but the optimized Distance encoding remains competitive, with runtimes close to *CRE*'s. In other graphs (e.g.,

171, 197, 249, 254, 491, 506, 522, 526, 529), *CRE* achieves the best performance, often significantly (e.g., 0.999 seconds in graph 249 vs. 33.517 seconds for optimized). However, the optimized Distance encoding's ability to outperform *CRE* in several instances and maintain competitiveness in others highlights its robustness across diverse graph structures.

The outperforming and competitive result in certain cases of Distance encoding and *CRE* should be due to its strong unit propagation properties and better alignment with modern SAT solver heuristics. Its sequential structure allows for more predictable and effective clause learning, making it easier for solvers to prune the search space. Additionally, Distance encoding avoids the complexity of modular arithmetic in *CRE*, resulting in simpler clauses that are often easier for solvers to process and resolve, especially on graphs where cycle lengths do not factor well into small primes.

The number of variables and clauses generated by an encoding directly impacts the solver's performance for HCP, as they determine the complexity of the resulting SAT instance. Tables 4.4 and 4.5 present the number of variables and clauses, respectively, for the non-optimized and optimized versions of the Distance encoding, as well as the *CRE*.

Table 4.4: Comparison number of variables

graph#	**BDE**	**ODE**	**CRE**
48	151,086	154,466	4,457
162	973,539	988,083	825,834
171	1,157,352	1,177,272	6,975
197	1,657,260	1,645,380	8,319
223	2,235,618	2,216,214	14,911
237	2,519,532	2,498,868	10,335
249	2,793,494	2,771,682	9,351
252	2,840,604	2,821,740	11,007
254	2,874,494	2,855,510	9,495
255	2,881,296	2,862,288	22,639
424	6,781,500	6,751,908	29,255
446	7,274,665	7,243,981	29,935
470	8,318,640	8,280,280	37,061
491	8,941,536	8,901,720	19,911
506	9,686,352	9,644,856	20,751
522	10,306,080	10,263,240	21,423
526	10,616,928	10,579,632	21,759
529	10,774,080	10,736,496	21,927

Table 4.5: Comparison number of clauses

graph#	BDE	ODE	CRE
48	6,904,812	6,878,423	44,481
162	44,596,439	44,421,546	5,775,807
171	54,140,541	53,824,165	44,840
197	100,620,150	100,478,589	53,480
223	136,763,380	136,772,414	118,210
237	155,341,574	155,099,475	66,440
249	173,072,478	172,798,650	51,434
252	176,191,446	175,920,391	70,760
254	178,437,068	178,162,739	52,226
255	178,584,172	178,610,615	193,057
424	396,380,435	396,380,778	240,615
446	426,159,137	426,150,026	247,657
470	489,358,119	489,277,632	309,121
491	527,993,773	527,101,132	128,000
506	573,460,001	572,484,285	133,400
522	611,188,981	610,143,945	137,720
526	630,492,093	629,424,343	139,880
529	640,256,069	639,170,797	140,960

Table 4.4 shows that the Distance encoding, in both its non-optimized and optimized forms, consistently generates a significantly higher number of variables compared to *CRE*. Table 4.5 reveals that the optimized Distance encoding typically reduces the clause count slightly compared to the non-optimized version. A more significant disparity, with *CRE* generating far fewer clauses than both versions of the Distance encoding in most graphs. This substantial decrease in clause count is due to *CRE*'s use of incremental SAT with small cycle sizes. By focusing on small cycle sizes, *CRE* minimizes the complexity of each SAT instance, allowing the solver to process constraints more efficiently. However, this advantage comes with trade-offs[15]; *CRE*'s incremental SAT approach requires multiple solver calls to construct the full cycle, which can increase overall runtime in graphs where intermediate cycles fail to extend to a Hamiltonian cycle. Additionally, *CRE* does not guarantee effectiveness across all graphs, as its performance depends on the graph's structure and the feasibility of finding small cycles that can be extended, whereas the Distance encoding provides a more consistent approach and produces results competitive with *CRE*.

To evaluate the effectiveness of four proposed methods for solving HCP, experiments were conducted to compare the performance of their optimized and non-

optimized versions. The four line charts (Figures 1–4) compare the runtimes of optimized and non-optimized versions of four techniques for solving HCP across a set of benchmark graphs, excluding 3 timeout graphs (424, 446, 470). Each chart evaluates a specific optimization strategy: preprocessing, symmetry breaking, start vertex optimization, and at-most-one encoding approach (*PBLib* vs. *Hybrid*). The x-axis represents the graph instances, and the y-axis represents the runtime in seconds. The analysis reveals the effectiveness of each optimization technique in improving solver performance.

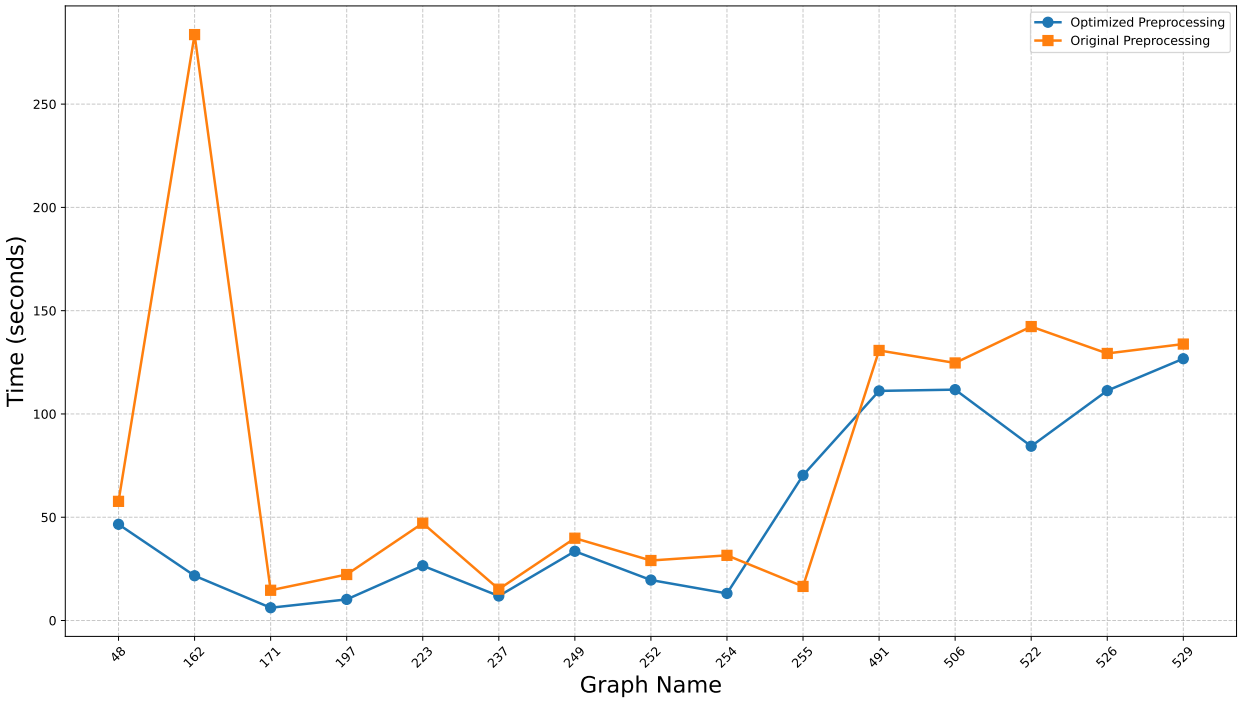


Figure 4.1: Effect of optimized preprocessing

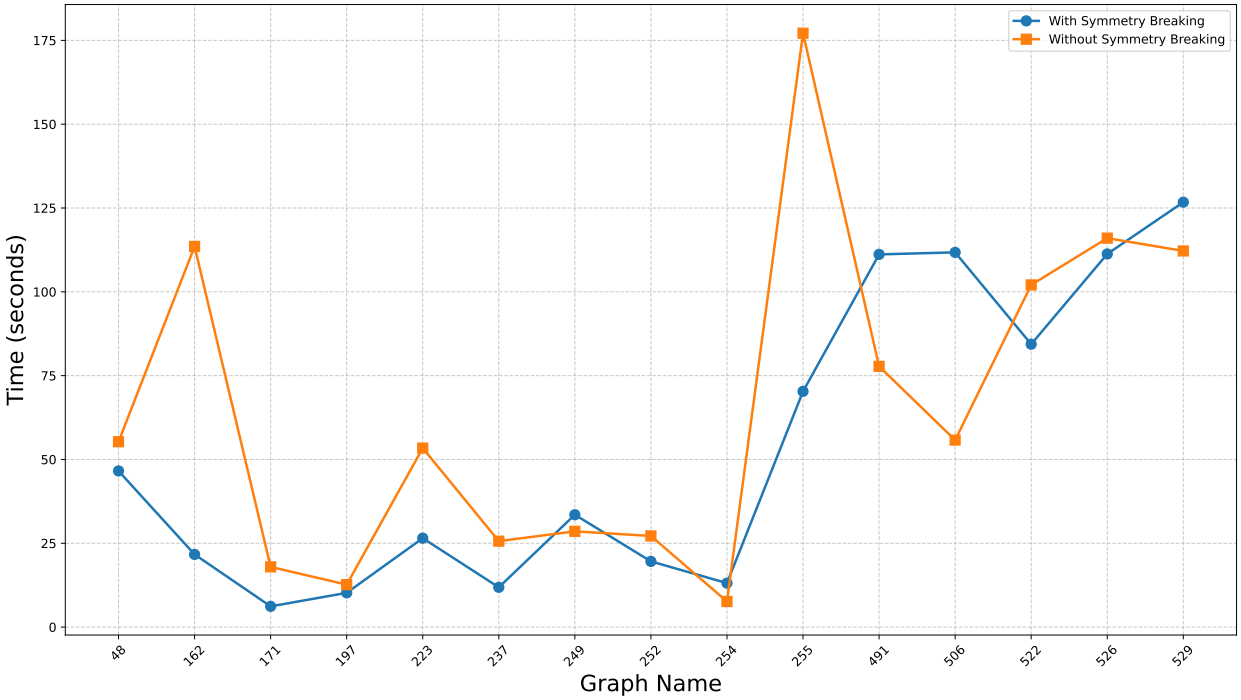


Figure 4.2: Effect of using symmetry breaking

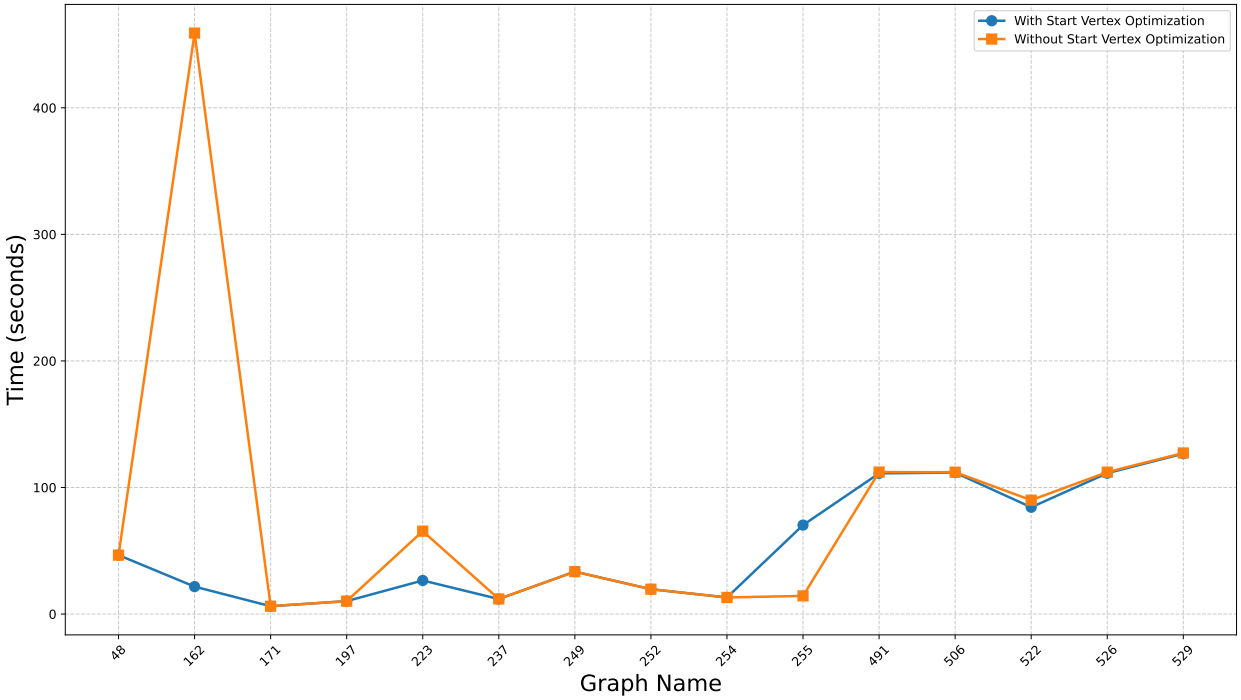


Figure 4.3: Effect of using start vertex

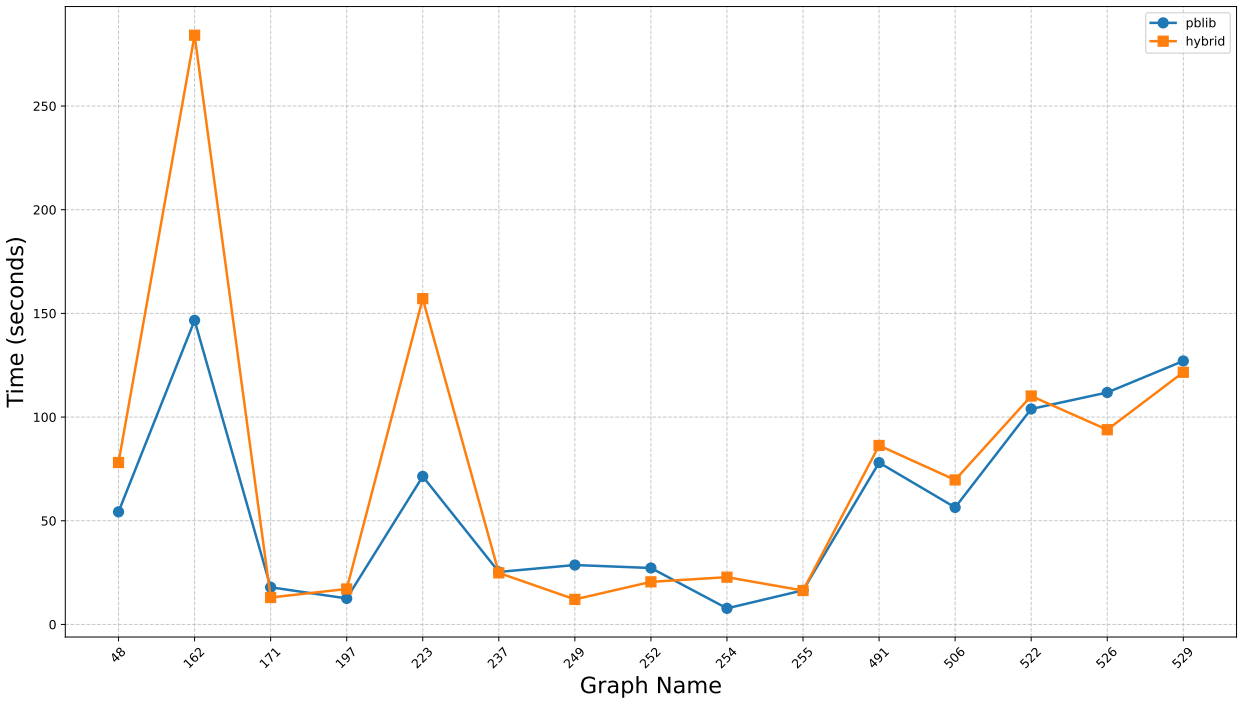


Figure 4.4: Effect of using PBLib

Figure 4.1 (Comparison of preprocessing times) demonstrates that optimized preprocessing significantly outperforms the original preprocessing across nearly all graphs. The optimized version (blue line) consistently achieves lower runtimes, with notable reductions in graphs like 162 (from approximately 280 seconds to 50 seconds) and 197 (from 80 seconds to 20 seconds). Even in graphs with smaller differences, such as 237 and 252, the optimized preprocessing maintains an advantage, reducing runtimes by around 20–30 seconds. The only exception is graph 255, where the original preprocessing is slightly faster (around 90 seconds vs. 100 seconds), possibly due to graph-specific properties that favor simpler preprocessing.

Figure 4.2 (Comparison of symmetry breaking vs. no-symmetry breaking) reveals that incorporating symmetry breaking significantly enhances performance across nearly all graphs. Usage of symmetry breaking adds clauses to the solver, but reduces the search space by eliminating the two-dimensionality of circles. The symmetry breaking approach (blue line) reduces runtimes substantially in graphs like 162 (from 175 seconds to 50 seconds) and 491 (from 60 seconds to 20 seconds). The improvement is particularly pronounced in graphs with high symmetry, such as 223 and 506, where runtimes drop by 50–70 seconds. The only graph where symmetry breaking does not provide a clear advantage is 255, with a minor increase (100 seconds vs. 90 seconds), likely due to low symmetry in its structure. Overall, symmetry breaking proves to be a highly effective optimization strategy for HCP.

Figure 4.3 (Comparison of start vertex optimization vs. no optimization) highlights the selective effectiveness of start vertex optimization. This technique, which prioritizes a starting vertex based on graph properties, performs better in specific graphs where the minimum degree vertex differs from vertex 1, such as graphs 171, 197, 237, and 252. In these cases, the optimized version (blue line) reduces runtimes significantly—for example, in graph 171 (from 50 seconds to 10 seconds) and graph 252 (from 80 seconds to 20 seconds). However, in graphs like 162 and 255, where the minimum degree vertex aligns with vertex 1, the optimization yields minimal improvement or even increases runtime (e.g., graph 255: 50 seconds without optimization vs. 60 seconds with optimization), indicating that start vertex optimization's benefits are highly graph-dependent.

Figure 4.4 (Comparison of *PBLib* vs. *Hybrid*) shows that using *PBLib* for encoding at-most-one constraints slightly decreases solver time compared to a hybrid approach across most graphs. The *PBLib* usage method (blue line) consistently achieves modest runtime reductions, such as in graph 162 (from 250 seconds to 200 seconds) and graph 491 (from 80 seconds to 60 seconds). The improvements are generally small, averaging 10–20 seconds across the dataset, but are consistent, with the exception of graph 255, where the hybrid approach is marginally faster (90 seconds vs. 100 seconds). This suggests that *PBLib*'s efficient handling of pseudo-Boolean constraints provides a slight but reliable advantage in most cases.

In summary, the line charts illustrate that optimized preprocessing and symmetry breaking are the most impactful techniques, consistently reducing runtimes across almost all graphs by simplifying the problem structure and eliminating redundant solutions. Start vertex optimization offers significant benefits but only for specific graphs where the minimum degree vertex differs from vertex 1, making it a more situational optimization. The use of *PBLib* provides a slight but consistent reduction in solver time, indicating its reliability for encoding HCP constraints. These findings underscore the importance of tailored optimization strategies in improving solver efficiency for HCP, with preprocessing and symmetry breaking emerging as the most universally effective approaches in this benchmark set.

Conclusion

This research has made significant contributions to solving the Hamiltonian Cycle Problem by leveraging SAT-based encodings, specifically enhancing the Distance encoding framework introduced by Zhou et al. (2020) [36]. The primary contributions lie in the development and implementation of four optimization techniques: optimized preprocessing, start vertex optimization, usage PBLib for cardinality constraint, and symmetry breaking. Optimized preprocessing reduces the search space by pruning redundant vertices and edges, start vertex optimization prioritizes specific vertices to guide the solver, PBLib improves the handling of pseudo-Boolean constraints, and symmetry breaking eliminates redundant solutions to enhance efficiency. These techniques were systematically evaluated through 16 configurations, providing a comprehensive understanding of their individual and combined effects on solver performance, variable counts, and clause counts.

Experimental results demonstrate that the optimized Distance encoding consistently outperforms the original Distance encoding across most benchmark graphs, achieving significant runtime reductions. It also proves competitive with the Chinese Remainder Encoding, surpassing it in several graphs while remaining close in others, despite CRE's advantage in clause compactness. Among the proposed optimizations, optimized preprocessing and symmetry breaking exhibit the most substantial impact, reducing runtimes significantly across nearly all graphs. Start vertex optimization and using PBLib yield positive results, with notable improvements in specific graphs, but their effectiveness is sometimes graph-dependent, particularly for start vertex optimization, which excels when the minimum degree vertex differs from vertex 1. These findings highlight the robustness of the optimized Distance encoding and the varying applicability of the proposed techniques.

Looking ahead, future research aims to further optimize the Distance encoding by exploring advanced techniques, such as dynamic preprocessing or hybrid encodings that combine the strengths of Distance encoding and CRE. Experiments on larger graphs and diverse benchmarks, such as the knight's tour dataset, will provide deeper

insights into the scalability and generalizability of the proposed methods. Additionally, applying symmetry breaking and other optimization strategies to the Chinese Remainder Encoding method could mitigate its limitations, such as the overhead of multiple SAT calls, potentially enhancing its performance across a broader range of graphs. These directions promise to advance the application of SAT technology in solving complex graph-theoretic problems like HCP, building on the foundation established in this thesis.

References

[1] *FHCP Challenge Set / Flinders Hamiltonian Cycle Project — sites.flinders.edu.au*, https://sites.flinders.edu.au/flinders-hamiltonian-cycle-project/fhcp-challenge-set/, [Accessed 07-05-2025].

[2] Takanori Akiyama, Takao Nishizeki, and Nobuji Saito, *Np-completeness of the hamiltonian cycle problem for bipartite graphs*, Journal of Information processing **3** (1980), no. 2, 73–76.

[3] Gilles Audemard and Laurent Simon, *Glucose in the sat 2014 competition*, Proceedings of SAT Competition **2014** (2014), 31.

[4] Armin Biere, *Lingeling, plingeling, picosat and precosat at sat race 2010*, (2010).

[5] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt, *Cadical 2.0*, International Conference on Computer Aided Verification, Springer, 2024, pp. 133–152.

[6] Jean-Charles Billaut, Federico Della Croce, Fabio Salassa, and Vincent t'Kindt, *When shop scheduling meets dominoes, eulerian and hamiltonian paths*, 13th Workshop on Models and Algorithms for Planning and Scheduling Problems (MAPSP 2017), 2017.

[7] Jingchao Chen, *A new sat encoding of the at-most-one constraint*, Proc. constraint modelling and reformulation (2010), 8.

[8] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith, *Counterexample-guided abstraction refinement*, Computer Aided Verification: 12th International Conference, CAV 2000, Chicago, IL, USA, July 15-19, 2000. Proceedings 12, Springer, 2000, pp. 154–169.

[9] James Cooper and Radu Nicolescu, *The hamiltonian cycle and travelling salesman problems in cp systems*, Fundamenta Informaticae **164** (2019), no. 2-3, 157–180.

[10] Nils Froleyks, Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda, *Sat competition 2020*, Artificial Intelligence **301** (2021), 103572.

[11] Aarti Gupta, Malay K Ganai, and Chao Wang, *Sat-based verification methods and applications in hardware verification*, International School on Formal Methods for the Design of Computer, Communication and Software Systems, Springer, 2006, pp. 108–143.

[12] Frank Gurski, *Dynamic programming algorithms on directed cographs*, Statistics, Optimization & Information Computing **5** (2017), no. 1, 35–44.

[13] Michael Haythorpe, *Fhcp challenge set: The first set of structurally difficult instances of the hamiltonian cycle problem*, arXiv preprint arXiv:1902.10352 (2019).

[14] Michael Haythorpe and Andrew Johnson, *Change ringing and hamiltonian cycles: The search for erin and stedman triples*, arXiv preprint arXiv:1702.02623 (2017).

[15] Marijn JH Heule, *Chinese remainder encoding for hamiltonian cycles*, Theory and Applications of Satisfiability Testing–SAT 2021: 24th International Conference, Barcelona, Spain, July 5-9, 2021, Proceedings 24, Springer, 2021, pp. 216–224.

[16] Hong Huang and John Copeland, *Hamiltonian cycle protection: A novel approach to mesh wdm optical network protection*, 2001 IEEE Workshop on High Performance Switching and Routing (IEEE Cat. No. 01TH8552), IEEE, 2001, pp. 31–35.

[17] Cor AJ Hurkens and Gerhard J Woeginger, *On the nearest neighbor rule for the traveling salesman problem*, Operations Research Letters **32** (2004), no. 1, 1–4.

[18] Franjo Ivančić, Zijiang Yang, Malay K Ganai, Aarti Gupta, and Pranav Ashar, *Efficient sat-based bounded model checking for software verification*, Theoretical Computer Science **404** (2008), no. 3, 256–274.

[19] Kazuo Iwama, *Sat-variable complexity of hard combinatorial problems*, Proc. IFIP 13th World Computer Congress, 1994, 1994, pp. 253–258.

[20] Andrew Johnson, *Quasi-linear reduction of hamiltonian cycle problem (hcp) to satisfiability problem (sat)*, IP. com Disclosure Number IPCOM000237123D (2014).

[21] Jin-Woo Jung and Hyun-Min Park, *A study on hamiltonian cycle-based path planning for multiple robot-single station docking problem*, Advanced Science Letters **22** (2016), no. 11, 3352–3355.

[22] Srihder Kaminanai, *Finding hamiltonian cycles*, (2005).

[23] Will Klieber and Gihwon Kwon, *Efficient cnf encoding for selecting 1 from n objects*, Proc. International Workshop on Constraints in Formal Verification, 2007, p. 14.

[24] I Mayer, *Towards a "chemical" hamiltonian*, International Journal of Quantum Chemistry **23** (1983), no. 2, 341–363.

[25] Shidharshekhar Neelannavar, Meenakshi H, Madhu N R, and Reshma R, *Hamiltonian cycle and hamiltonian path and its applications–a review*, Journal of Graph Theory Applications **12** (2023), no. 3, 45–52, Department of Mathematics, R L Jalappa Institute of Technology, Doddaballapur, India.

[26] Tobias Philipp and Peter Steinke, *Pblib–a library for encoding pseudo-boolean constraints into cnf*, International Conference on Theory and Applications of Satisfiability Testing, Springer, 2015, pp. 9–16.

[27] SEPARATE DECISION QUEUE, *Cadical at the sat race 2019*, SAT RACE **2019** (2019), 8.

[28] M Sohel Rahman and Mohammad Kaykobad, *On hamiltonian cycles and hamiltonian paths*, Information Processing Letters **94** (2005), no. 1, 37–41.

[29] Joeri Sleegers and Daan van den Berg, *Backtracking (the) algorithms on the hamiltonian cycle problem*, arXiv preprint arXiv:2107.00314 (2021).

[30] Takehide Soh, Daniel Le Berre, Stéphanie Roussel, Mutsunori Banbara, and Naoyuki Tamura, *Incremental sat-based method with native boolean cardinality handling for the hamiltonian cycle problem*, Logics in Artificial Intelligence: 14th European Conference, JELIA 2014, Funchal, Madeira, Portugal, September 24-26, 2014. Proceedings 14, Springer, 2014, pp. 684–693.

[31] Niklas Sörensson, *Minisat 2.2 and minisat++ 1.1*, A short description in SAT Race **2010** (2010).

[32] Ian Stewart, *Knight's tours*, Scientific American **276** (1997), no. 4, 102–104.

[33] Bernhard Thalheim, *Fundamentals of cardinality constraints*, International Conference on Conceptual Modeling, Springer, 1992, pp. 7–23.

[34] Morgane Thomas-Chollier, Olivier Sand, Jean-Valery Turatsinze, Rekin's Janky, Matthieu Defrance, Eric Vervisch, Sylvain Brohee, and Jacques van Helden, *Rsat: regulatory sequence analysis tools*, Nucleic acids research **36** (2008), no. suppl_2, W119–W127.

[35] Ping Zhang, *Hamiltonian extension*, Graph Theory: Favorite Conjectures and Open Problems-1 (2016), 17–30.

[36] Neng-Fa Zhou, *In pursuit of an efficient sat encoding for the hamiltonian cycle problem*, International Conference on Principles and Practice of Constraint Programming, Springer, 2020, pp. 585–602.

[37] ———, *Encoding the hamiltonian cycle problem into sat based on vertex elimination (short paper)*, 30th International Conference on Principles and Practice of Constraint Programming (CP 2024), Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024, pp. 40–1.

Appendix

This appendix provides a comprehensive collection of all experimental results conducted as part of the research on HCP using SAT-based encodings, includes both distance encoding and CRE experiments. The tables detail the runtime performance, number of variables, and number of clauses for various encoding methods across the benchmark graph set.

For distance encoding, there are 16 distinct configurations, labeled from a to p, derived from the combination of four proposed optimization methods: optimized preprocessing, start vertex technique, PBLib encoding, and symmetry breaking. Each configuration represents a unique combination of these techniques, applied either individually or together. For instance, configuration f might employ only optimized preprocessing, while c could integrate all four optimizations.

For CRE, this thesis performs and re-experiments the results on the same environment to ensure objectivity when comparing with distance encoding. The experiment evaluates CRE's performance across a benchmark graph set, with results summarized in a table where the first column lists the graph names (e.g., 48, 162, 171, etc.), and the first row specifies the values of k, the cycle size used to determine the existence of a Hamiltonian cycle (k=2, 6, 12, 60, 105, 420), allowing for an analysis of how different cycle sizes impact CRE's efficiency in finding Hamiltonian cycles across diverse graph structures.

Table 5.1: All 16 configurations by combining four optimization techniques

	a	**b**	**c**	**d**	**e**	**f**	**g**	**h**	**i**	**j**	**k**	**l**	**m**	**n**	**o**	**p**
PBLib			x	x			x	x			x	x			x	x
Preprocessing	x	x	x	x	x	x	x	x								
Start vertex	x		x		x		x		x		x		x		x	
Symmetry breaking	x	x	x	x					x	x	x	x				

Table 5.2: Runtime results of configurations a to h (in seconds)

graph#	**a**	**b**	**c**	**d**	**e**	**f**	**g**	**h**
48	61.644	61.609	46.573	46.559	78.185	78.091	55.284	54.297
162	17.349	*timeout*	21.672	459.059	183.304	284.162	113.54	146.637
171	7.829	7.831	6.182	6.215	13.021	12.984	17.953	17.917
197	15.72	15.627	10.213	10.203	17.07	17.099	12.663	12.546
223	29.004	85.676	26.523	65.52	86.365	157.107	53.386	71.395
237	22.082	21.939	11.862	11.914	25.055	24.859	25.636	25.312
249	16.323	16.12	33.517	33.486	11.546	12.072	28.56	28.65
252	34.082	33.973	19.596	19.657	20.301	20.528	27.183	27.217
254	36.378	36.44	13.11	13.096	22.548	22.795	7.63	7.751
255	24.856	19.258	70.305	14.34	17.329	16.351	177.14	16.454
424	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*
446	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*
470	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*
491	115.805	120.606	111.16	112.186	90.193	86.31	77.762	78.038
506	105.623	105.921	111.77	112.089	71.214	69.686	55.78	56.428
522	103.005	103.202	84.375	89.988	110.133	110.16	102.059	103.91
526	119.709	98.543	111.288	112.141	93.539	93.893	116.015	111.857
529	103.784	101.915	126.74	127.26	121.721	121.617	112.195	127.05
Avg	54.213	59.190	53.659	82.248	64.102	75.181	65.519	59.031

Table 5.3: Runtime results of configurations i to p (in seconds)

graph#	**i**	**j**	**k**	**l**	**m**	**n**	**o**	**p**
48	42.186	42.338	36.68	37.229	57.807	57.693	53.179	52.791
162	38.341	164.325	26.314	109.531	154.077	283.664	455.392	391.988
171	17.8	17.831	10.27	10.305	14.583	14.646	6.419	6.389
197	22.205	22.303	11.1	11.316	22.359	22.269	26.127	25.983
223	47.48	24.185	75.604	118.852	40.203	47.119	61.02	19.813
237	24.811	25.023	22.452	22.651	15.186	15.171	27.64	27.263
249	39.3	38.139	36.102	37.198	39.568	39.848	17.14	17.235
252	32.666	32.846	31.688	31.751	29.127	29.047	38.21	38.124
254	37.773	38.073	29.786	29.89	31.433	31.552	17.036	17.028
255	38.417	29.932	55.855	17.955	246.514	16.494	115.601	35.109
424	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*
446	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*
470	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*	*timeout*
491	125.11	128.124	110.482	110.869	130.155	130.744	113.02	112.921
506	134.455	140.591	183.768	186.177	124.858	124.69	120.951	121.074
522	133.601	140.924	119.746	121.997	144.292	142.307	145.195	145.59
526	151.638	162.015	143.852	147.856	132.194	129.293	142.466	146.718
529	317.16	335.493	174.586	177.745	136.516	133.838	152.672	176.513
Avg	80.196	89.476	71.219	78.088	87.925	81.225	99.471	88.969

Table 5.4: Number of variables in configurations a to h

graph#	a	b	c	d	e	f	g	h
48	151,086	151,086	154,466	154,466	151,086	151,086	154,466	154,466
162	973,539	973,539	988,083	988,083	973,539	973,539	988,083	988,083
171	1,157,352	1,157,352	1,177,272	1,177,272	1,157,352	1,157,352	1,177,272	1,177,272
197	1,657,260	1,657,260	1,645,380	1,645,380	1,657,260	1,657,260	1,645,380	1,645,380
223	2,235,618	2,235,618	2,216,214	2,216,214	2,235,618	2,235,618	2,216,214	2,216,214
237	2,519,532	2,519,532	2,498,868	2,498,868	2,519,532	2,519,532	2,498,868	2,498,868
249	2,793,494	2,793,494	2,771,682	2,771,682	2,793,494	2,793,494	2,771,682	2,771,682
252	2,840,604	2,840,604	2,821,740	2,821,740	2,840,604	2,840,604	2,821,740	2,821,740
254	2,874,494	2,874,494	2,855,510	2,855,510	2,874,494	2,874,494	2,855,510	2,855,510
255	2,881,296	2,881,296	2,862,288	2,862,288	2,881,296	2,881,296	2,862,288	2,862,288
424	6,781,500	6,781,500	6,751,908	6,751,908	6,781,500	6,781,500	6,751,908	6,751,908
446	7,274,665	7,274,665	7,243,981	7,243,981	7,274,665	7,274,665	7,243,981	7,243,981
470	8,318,640	8,318,640	8,280,280	8,280,280	8,318,640	8,318,640	8,280,280	8,280,280
491	8,941,536	8,941,536	8,901,720	8,901,720	8,941,536	8,941,536	8,901,720	8,901,720
506	9,686,352	9,686,352	9,644,856	9,644,856	9,686,352	9,686,352	9,644,856	9,644,856
522	10,306,080	10,306,080	10,263,240	10,263,240	10,306,080	10,306,080	10,263,240	10,263,240
526	10,616,928	10,616,928	10,579,632	10,579,632	10,616,928	10,616,928	10,579,632	10,579,632
529	10,774,080	10,774,080	10,736,496	10,736,496	10,774,080	10,774,080	10,736,496	10,736,496

Table 5.5: Number of variables in configurations i to p

graph#	i	j	k	l	m	n	o	p
48	151,086	151,086	154,466	154,466	151,086	151,086	154,466	154,466
162	973,539	973,539	988,083	988,083	973,539	973,539	988,083	988,083
171	1,157,352	1,157,352	1,177,272	1,177,272	1,157,352	1,157,352	1,177,272	1,177,272
197	1,657,260	1,657,260	1,645,380	1,645,380	1,657,260	1,657,260	1,645,380	1,645,380
223	2,235,618	2,235,618	2,216,214	2,216,214	2,235,618	2,235,618	2,216,214	2,216,214
237	2,519,532	2,519,532	2,498,868	2,498,868	2,519,532	2,519,532	2,498,868	2,498,868
249	2,793,494	2,793,494	2,771,682	2,771,682	2,793,494	2,793,494	2,771,682	2,771,682
252	2,840,604	2,840,604	2,821,740	2,821,740	2,840,604	2,840,604	2,821,740	2,821,740
254	2,874,494	2,874,494	2,855,510	2,855,510	2,874,494	2,874,494	2,855,510	2,855,510
255	2,881,296	2,881,296	2,862,288	2,862,288	2,881,296	2,881,296	2,862,288	2,862,288
424	6,781,500	6,781,500	6,751,908	6,751,908	6,781,500	6,781,500	6,751,908	6,751,908
446	7,274,665	7,274,665	7,243,981	7,243,981	7,274,665	7,274,665	7,243,981	7,243,981
470	8,318,640	8,318,640	8,280,280	8,280,280	8,318,640	8,318,640	8,280,280	8,280,280
491	8,941,536	8,941,536	8,901,720	8,901,720	8,941,536	8,941,536	8,901,720	8,901,720
506	9,686,352	9,686,352	9,644,856	9,644,856	9,686,352	9,686,352	9,644,856	9,644,856
522	10,306,080	10,306,080	10,263,240	10,263,240	10,306,080	10,306,080	10,263,240	10,263,240
526	10,616,928	10,616,928	10,579,632	10,579,632	10,616,928	10,616,928	10,579,632	10,579,632
529	10,774,080	10,774,080	10,736,496	10,736,496	10,774,080	10,774,080	10,736,496	10,736,496

Table 5.6: Number of clauses in configurations a to h

graph#	a	b	c	d	e	f	g	h
48	6,902,759	6,902,759	6,878,423	6,878,423	6,902,421	6,902,421	6,878,085	6,878,085
162	44,601,528	44,600,980	44,421,546	44,420,998	44,600,619	44,600,071	44,420,637	44,420,089
171	54,023,365	54,023,365	53,824,165	53,824,165	54,022,369	54,022,369	53,823,169	53,823,169
197	100,454,829	100,454,829	100,478,589	100,478,589	100,453,641	100,453,641	100,477,401	100,477,401
223	136,755,782	136,755,730	136,772,414	136,772,362	136,754,396	136,754,344	136,771,028	136,770,976
237	155,081,763	155,081,763	155,099,475	155,099,475	155,080,287	155,080,287	155,097,999	155,097,999
249	172,779,954	172,779,954	172,798,650	172,798,650	172,778,396	172,778,396	172,797,092	172,797,092
252	175,895,239	175,895,239	175,920,391	175,920,391	175,893,667	175,893,667	175,918,819	175,918,819
254	178,137,427	178,137,427	178,162,739	178,162,739	178,135,845	178,135,845	178,161,157	178,161,157
255	178,585,271	178,584,689	178,610,615	178,610,033	178,583,687	178,583,105	178,609,031	178,608,449
424	396,292,002	396,292,039	396,380,778	396,380,815	396,289,536	396,289,573	396,378,312	396,378,349
446	426,057,974	426,057,812	426,150,026	426,149,864	426,055,417	426,055,255	426,147,469	426,147,307
470	489,189,952	489,190,894	489,277,632	489,278,574	489,187,212	489,188,154	489,274,892	489,275,834
491	527,010,124	527,010,124	527,101,132	527,101,132	527,007,280	527,007,280	527,098,288	527,098,288
506	572,389,437	572,389,437	572,484,285	572,484,285	572,386,473	572,386,473	572,481,321	572,481,321
522	610,046,025	610,046,025	610,143,945	610,143,945	610,042,965	610,042,965	610,140,885	610,140,885
526	629,312,455	629,312,455	629,424,343	629,424,343	629,309,347	629,309,347	629,421,235	629,424,343
529	639,058,045	639,058,045	639,170,797	639,170,797	639,054,913	639,054,913	639,167,665	639,170,797

Table 5.7: Number of clauses in configurations i to p

graph#	i	j	k	l	m	n	o	p
48	6,905,150	6,905,150	6,880,814	6,880,814	6,904,812	6,904,812	6,880,476	6,880,476
162	44,599,704	44,597,348	44,419,722	44,417,366	44,598,795	44,596,439	44,418,813	44,416,457
171	54,141,537	54,141,537	53,942,337	53,942,337	54,140,541	54,140,541	53,941,341	53,941,341
197	100,621,338	100,621,338	100,645,098	100,645,098	100,620,150	100,620,150	100,643,910	100,643,910
223	136,767,658	136,764,766	136,784,290	136,781,398	136,766,272	136,763,380	136,782,904	136,780,012
237	155,343,050	155,343,050	155,360,762	155,360,762	155,341,574	155,341,574	155,359,286	155,359,286
249	173,074,036	173,074,036	173,092,732	173,092,732	173,072,478	173,072,478	173,091,174	173,091,174
252	176,193,018	176,193,018	176,218,170	176,218,170	176,191,446	176,191,446	176,216,598	176,216,598
254	178,438,650	178,438,650	178,463,962	178,463,962	178,437,068	178,437,068	178,462,380	178,462,380
255	178,597,154	178,585,756	178,622,498	178,611,100	178,595,570	178,584,172	178,620,914	178,609,516
424	396,382,901	396,376,767	396,471,677	396,465,543	396,380,435	396,380,435	396,469,211	396,468,216
446	426,161,694	426,168,146	426,253,746	426,260,198	426,159,137	426,159,137	426,251,189	426,250,123
470	489,360,859	489,384,907	489,448,539	489,450,627	489,358,119	489,358,119	489,445,799	489,444,814
491	527,996,617	527,996,617	528,087,625	528,087,625	527,993,773	527,993,773	528,084,781	528,084,781
506	573,462,965	573,462,965	573,557,813	573,557,813	573,460,001	573,460,001	573,554,849	573,554,849
522	611,192,041	611,192,041	611,289,961	611,289,961	611,188,981	611,188,981	611,286,901	611,286,901
526	630,495,201	630,495,201	630,607,089	630,607,089	630,492,093	630,492,093	630,603,981	630,607,089
529	640,259,201	640,259,201	640,371,953	640,371,953	640,256,069	640,256,069	640,368,821	640,371,953

Table 5.8: Runtime result of Chinese Remainder Encoding (in seconds)

graph#	2	6	12	60	105	420
48	0.044 ✗	6.111 ✗	12.276 ✗	22.688 ✗	46.857 ✓	85.107 ✓
162	31.595 ✗	15.295 ✓	19.872 ✓	33.49 ✓	35.533 ✓	38.164 ✓
171	0.001 ✗	1.317 ✗	0.916 ✓	3.611 ✓	3.96 ✓	5.97 ✓
197	0.017 ✗	1.216 ✗	3.14 ✓	9.884 ✓	6.241 ✓	12.876 ✓
223	0.026 ✗	0.673 ✗	8.54 ✗	72.746 ✓	7.612 ✓	48.504 ✓
237	0.016 ✗	0.742 ✗	9.936 ✓	3.701 ✓	4.218 ✗	16.075 ✓
249	0.252 ✗	0.747 ✓	4.853 ✓	2.061 ✓	2.273 ✓	5.378 ✓
252	0.024 ✗	6.335 ✗	10.885 ✓	12.116 ✓	63.673 ✓	22.563 ✓
254	0.096 ✗	0.815 ✓	1.596 ✓	1.905 ✓	2.838 ✓	3.989 ✓
255	0.067 ✗	1.969 ✗	2.893 ✗	5.065 ✗	11.64 ✗	12.901 ✓
424	5.483 ✗	182.737 ✗	199.488 ✗	182.298 ✗	787.665 ✓	681.125 ✓
446	8.442 ✗	137.177 ✗	157.284 ✗	297.976 ✗	748.848 ✓	674.392 ✓
470	12.815 ✗	213.354 ✗	208.001 ✗	294.331 ✗	396.397 ✗	346.217 ✓
491	0.024 ✗	17.043 ✗	6.744 ✓	85.594 ✓	31.765 ✓	80.732 ✓
506	0.051 ✗	6.725 ✗	17.574 ✓	50.981 ✓	33.657 ✓	73.193 ✓
522	0.018 ✗	3.615 ✗	30.309 ✓	31.295 ✓	142.032 ✗	126.433 ✓
526	0.04 ✗	20.133 ✗	19.244 ✓	23.521 ✓	23.38 ✓	153.748 ✓
529	0.192 ✗	4.957 ✗	94.753 ✓	58.999 ✓	95.693 ✓	84.798 ✓

Table 5.9: Number of variables of Chinese Remainder Encoding

graph#	2	6	12	60	105	420
48	2,091	2,767	3,105	4,119	4,457	5,133
162	824,016	825,834	826,743	829,470	830,379	832,197
171	3,987	5,979	6,975	9,963	10,959	12,951
197	4,755	7,131	8,319	11,883	13,071	15,447
223	6,595	9,367	10,753	14,911	16,297	19,069
237	5,907	8,859	10,335	14,763	16,239	19,191
249	6,235	9,351	10,909	15,583	17,141	20,257
252	6,291	9,435	11,007	15,723	17,295	20,439
254	6,331	9,495	11,077	15,823	17,405	20,569
255	8,383	11,551	13,135	17,887	19,471	22,639
424	11,993	16,925	19,391	26,789	29,255	34,187
446	12,036	17,150	19,707	27,378	29,935	35,049
470	12,401	17,881	20,621	28,841	31,581	37,061
491	11,379	17,067	19,911	28,443	31,287	36,975
506	11,859	17,787	20,751	29,643	32,607	38,535
522	12,243	18,363	21,423	30,603	33,663	39,783
526	12,435	18,651	21,759	31,083	34,191	40,407
529	12,531	18,795	21,927	31,323	34,455	40,719

Table 5.10: Number of clauses of Chinese Remainder Encoding

graph#	**2**	**6**	**12**	**60**	**105**	**420**
48	9,038	18,674	24,871	41,042	44,481	53,779
162	3,296,052	5,775,807	7,428,370	11,561,597	12,388,789	14,867,635
171	13,962	32,888	44,840	76,714	83,688	101,618
197	16,650	39,224	53,480	91,498	99,816	121,202
223	23,354	51,950	70,089	118,210	128,667	155,877
237	20,682	48,728	66,440	113,674	124,008	150,578
249	21,830	51,434	70,130	119,988	130,896	158,942
252	22,026	51,896	70,760	121,066	132,072	160,370
254	22,166	52,226	71,210	121,836	132,912	161,390
255	30,006	65,172	87,559	146,696	159,475	193,057
424	44,214	97,554	131,469	221,190	240,615	291,489
446	45,260	100,227	135,166	227,629	247,657	300,067
470	45,686	102,528	138,595	234,244	255,019	309,121
491	39,834	93,872	128,000	219,010	238,920	290,114
506	41,514	97,832	133,400	228,250	249,000	302,354
522	42,858	101,000	137,720	235,642	257,064	312,146
526	43,530	102,584	139,880	239,338	261,096	317,042
529	43,866	103,376	140,960	241,186	263,112	319,490